HE-OFT: Privacy-Preserving One-Shot Federated Fine-Tuning of Pretrained Models under Homomorphic Encryption

Halil İbrahim Kanpak, Sinem Sav, Alptekin Küpçü

***Abstract*—Organizations adapt pretrained models to private data by fine-tuning, and parties with complementary data over the same task would each gain from a jointly adapted model, yet cannot pool their data to build one. Federated learning offers the collaboration but concentrates its privacy risk at training time: gradients and per-round updates permit reconstruction of training data and strong membership inference, and these training-time attacks are substantially more effective than attacks against the final released model. Existing protections pull against one another: multi-round cryptographic training is costly, and one-shot protocols either expose their single contribution or noise it, degrading the model they release. We present HE-OFT (Homomorphically Encrypted One-shot Fine-Tuning), a federated fine-tuning protocol that eliminates the training-time attack surface rather than perturbing it. Each client fine-tunes a small low-rank adapter on a frozen pretrained backbone in plaintext and uploads a single encrypted parameter displacement; the server's only operation is a sample-weighted linear combination under multiparty CKKS, a scheme chosen for its threshold decryption and packed real-number arithmetic rather than its circuit depth: the combination runs at multiplicative depth one and is decrypted jointly by a threshold of clients. Fine-tuning is what makes this affordable: the frozen backbone supplies the shared frame a one-shot linear aggregation needs under heterogeneous data, and freezing the adapter's down-projection makes the encrypted sum of the trained factors exactly the weighted average of the clients' weight updates, so the encrypted object is a few megabytes regardless of the backbone and the cryptographic layer adds no accuracy cost. Because a server computing under encryption cannot read its inputs, it emits several depth-one candidate aggregates and the clients select among them after decryption, which improves the released model under heterogeneity and neutralizes a single poisoning client at no homomorphic cost. Across vision and language tasks, from frozen RoBERTa and ViT to a half-billion-parameter language model, the method produces a common model stronger than any single client trains alone, within 0.01 of the same adapter fine-tuned on the pooled data when the data are near-uniform and 0.06–0.23 behind it at moderate label skew (α=0.1). A real multiparty CKKS implementation reproduces the plaintext aggregate within the approximation bounds of CKKS, and the one remaining surface, the released model, shows membership inference at or near chance on all but the largest label spaces, where it is mildly elevated and still far below the training-time attacks the protocol removes.**

***Index Terms*—Federated learning, one-shot federated learning, federated fine-tuning, homomorphic encryption, multiparty computation, privacy-preserving machine learning.**

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)

Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).

Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).

Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).

Manuscript received XX XX, XXXX; revised XX XX, XXXX.

I. Introduction

We target the setting that now dominates applied machine learning: organizations that adapt a public pretrained backbone to their own data by fine-tuning [1], [2]. Often several such organizations hold complementary data over the same task — different patient populations, fraud patterns, or customer bases — and each would obtain a better model from their combined data than from its own. Legal, competitive, and privacy constraints prevent them from pooling that data [3], [4]: data-protection law strictly restricts sharing confidential records across institutions — imposing data-minimisation and cross-border-transfer limits, and for health data a de-identification bar before any disclosure — under regimes from the EU GDPR and U.S. HIPAA to Turkey's KVKK and a widening set worldwide [5], [6], [7], [8], [9], and in high-risk domains such as health, biometrics, and credit the trained model is itself a regulated artifact [10]. Handing the data, or the model, to whoever operates the fine-tuning infrastructure raises the same concern [11]. What such parties need is fine-tuning as a service with a guarantee: a server that combines their local adaptations into one shared model while provably learning nothing about the data behind them. Providing that service securely is the problem this paper solves.

The reason the guarantee is hard to provide is that collaborative training leaks most *while it runs*. Federated learning exchanges model updates instead of data, but the updates themselves are the vulnerability: a gradient permits reconstruction of the training batch that produced it [12], [13], an observer of the per-round updates mounts membership inference far stronger than anything possible against the final model alone [14], [15], [16], and a server free to choose the weights it sends can harvest client inputs outright [17], [18]. Attacks against the released model after training, by contrast, are comparatively weak: on the same model and dataset, membership inference that reaches 87% accuracy for a server observing federated updates drops to 54.5% against the final model alone [14], and we confirm on our own protocol, with the same loss-threshold and likelihood-ratio attacks [19], [20], that it sits at or near chance (section IV-G). The privacy problem of federated learning is therefore concentrated at *training time*: in the artifacts clients expose while the model is being built, and in how many times they expose them.

That concentration sets the design goal of this paper: remove the training-time attack surface entirely, rather than perturb it. We therefore take the one-shot route within federated learning, and ask what it takes to protect its single exchange cryptographically.

Two levers, applied together, remove it. Making the protocol *one-shot* — a single upload and a single download per client — reduces the number of exposed artifacts from hundreds of rounds to one. Encrypting that one artifact under *homomorphic encryption* reduces its plaintext exposure to zero. Prior work pulls one lever at a time. One-shot federated learning sends its single contribution in plaintext or protects it with differential privacy, which injects noise into the released model and provides a statistical rather than cryptographic guarantee. Homomorphically encrypted federated learning runs many rounds of encrypted training or secure aggregation, at a cost measured in hours and gigabytes; the direct route of training under encryption forces polynomial replacements of every activation and a deep, repeatedly bootstrapped circuit [21], which is precisely the cost we set out to avoid. We present **HE-OFT** (Homomorphically Encrypted One-shot Fine-Tuning), to our knowledge the first one-shot federated learning protocol to protect *deep-model* fine-tuning contributions under homomorphic encryption; the sole prior one-shot encrypted federation we are aware of is confined to a single-layer, closed-form learner [22], whereas HE-OFT protects the gradient-trained low-rank adaptation of a pretrained deep backbone — the regime the foundation models now in deployment occupy. Each client fine-tunes a small low-rank adapter and classifier head on a frozen public backbone in plaintext, uploads one encrypted parameter displacement, and the server performs a single linear combination, $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$, under multiparty CKKS at multiplicative depth one, after which a threshold of clients jointly decrypts the result. We adopt the multiparty scheme for its key structure rather than its homomorphic depth: no single party, in particular not the server, can decrypt anything (section III-C). The server never holds a plaintext contribution, a secret key, or the decrypted model; nothing data-derived leaves a client unencrypted.

Fine-tuning is not incidental to this design; it is what makes the design work. A one-shot linear aggregation must overcome the fact that independently trained models do not average well under heterogeneous data: from different starting points, their updates conflict and cancel. A frozen pretrained backbone removes the difficulty without an extra exchange. It fixes the representation every client sees, and every client starts its small trainable unit from the same public initialization, so the bounded displacements they upload live in one frame and add constructively — where prior one-shot methods must first buy this alignment with an extra exchange of data-derived summaries. Freezing the adapter's down-projection, a parameterization we adopt from FFA-LoRA [23], then makes the aggregation *exact*: the weighted sum of the trained factors equals the weighted sum of the weight updates, so the depth-one encrypted combination computes precisely the model the method calls for, and the entire encrypted object is a few tens of ciphertexts, a few megabytes, whatever the size of the backbone behind it.

One capability is lost when the server computes under encryption: it cannot see the data, so it cannot adapt its aggregation rule to it. We restore that adaptivity on the clients' side. The server emits several *candidate* aggregates, each a depth-one linear function of the same encrypted displacements (scaled, curvature- and coverage-reweighted, and leave-one-out variants), and after threshold decryption the clients select among them by a weighted vote on their local data. The vote improves the released model under heterogeneity and, with the leave-one-out candidates, neutralizes a poisoning client — at no additional homomorphic cost.

What remains exposed is the released model itself, which every participant holds in the clear when the protocol's purpose is to hand out a shared model. This surface is intrinsic to releasing such a model, so we measure it: membership inference against the released model, including by a fellow participant, stays at or near chance, rising only on the largest label space. Where a deployment cannot release the model at all — when the model, fine-tuned on confidential data, is itself a regulated or proprietary asset — the same one-shot aggregate is instead kept encrypted and answers queries under encryption, exposing only the labels a participant asks for (section III-H). Across text classification and vision tasks, from frozen RoBERTa and ViT backbones to a half-billion-parameter language model, HE-OFT produces a common model stronger than any single client trains alone and, outside the most skewed regimes, close to a centrally fine-tuned one; a real multiparty CKKS implementation reproduces the plaintext aggregate within the scheme's approximation bounds, with server computation in well under a second and about ten megabytes of communication per client.

Our contributions are as follows.

- We introduce, to our knowledge, the first one-shot federated learning protocol to protect *deep-model* fine-tuning contributions under homomorphic encryption (prior one-shot encrypted federation being limited to a single-layer, closed-form learner [22]): a single round whose only server operation is a linear aggregation of multiplicative depth one under multiparty CKKS, followed by threshold decryption (section III). The protocol eliminates the training-time attack surface — the strongest one federated learning exposes — rather than perturbing it.
- We identify the co-design that makes one-shot encryption affordable. Fine-tuning a frozen pretrained backbone supplies the shared frame a one-shot linear aggregation needs at no cost, and freezing the low-rank adapter's down-projection makes the encrypted aggregate exactly the weighted average of the clients' weight updates (sections III and III-D).
- We introduce encrypted multi-candidate release with client-side selection: the server emits several depth-one candidate aggregates, and the clients, after threshold decryption, select one by a weighted vote on their local data. This restores data-dependent aggregation choice to a server that cannot read its inputs, improves the released model under heterogeneity, and yields a measured robustness defense (sections III-G and IV-D).
- We add a second disclosure setting: rather than releasing

the aggregate, the protocol can keep it encrypted and answer queries at multiplicative depth one, so the model itself is never revealed and a participant obtains only labels — a confidential-model serving mode for a one-shot, depth-one aggregate that we are not aware of in prior federated learning (section III-H).
- We validate both sides of the privacy design: the encrypted aggregation end to end in a real multiparty CKKS implementation at a few megabytes per client (section IV-F), and the remaining inference-time surface with membership inference against the released model, under external and fellow-participant adversaries (section IV-G).

Our implementation — the training and aggregation pipeline, the Lattigo protocol, and every experiment configuration in this paper — is available at **[TODO: link the anonymized repository before submission]**.

II. Related Work

Our setting draws on four lines of work: the study of what federated learning leaks and when; the cryptographic tools that protect training; one-shot federated learning and its differentially private variants; and federated fine-tuning of parameter-efficient adapters. We take them in that order, because the first motivates the design and the rest position it: the leakage literature shows that the decisive attack surface is the one exposed *while training runs*, and no existing line removes that surface in a single round without either noise or multi-round cryptography.

A. Leakage in Federated Learning: Training Time versus Inference Time

Federated learning began as an iterative procedure: a server repeatedly broadcasts a global model, clients improve it on their local data, and the server averages the returned updates [24]. Every round of this loop reveals a fresh artifact computed on private data, and a decade of attacks has established that these training-time artifacts are the most damaging thing a federated protocol exposes. A gradient permits reconstruction of the batch that produced it, pixel-accurate on images [12] and at high resolution even from trained deep networks, where averaging over a hundred images or a hundred local steps still leaves individual inputs recoverable [13]. An observer of the per-round updates infers membership and unintended properties of another client's data: with the update stream of a federated run on CIFAR-100, a passive server reaches 79.2% membership accuracy and an active one 87.3%, against 54.5% for a black-box attack on the same final model, the repeated per-round updates being, in the source's words, the key factor boosting the attack [14]; accuracy grows with the number of rounds observed [16], and update-level membership inference has reached 0.99 precision at perfect recall [15]. A server free to choose the weights it broadcasts does not need inference at all: crafted weights harvest verbatim copies of client inputs, through mini-batches of a hundred points and even through a secure-aggregation layer [17], [18]. Federated distillation, which shares predictions rather than gradients, was hoped to sidestep this, but its training-time artifacts leak too: paired-logit inversion recovers images from shared logits [25], and membership can be inferred from distillation outputs and public-set usage [26], [27], [28].

Attacks that see only the *released* model are consistently weaker, and the field's central survey draws exactly this line, distinguishing adversaries who see the intermediate iterates from those who see only the final model [29]. Membership inference against a finished model is a mature literature [30], [19], [31], and against well-generalized models its success concentrates at low false-positive-rate operating points rather than in bulk [20]; model inversion needs released confidences and strong priors [32], [33]. The asymmetry has a simple reason: training artifacts are high-dimensional views of a specific client's raw data taken repeatedly during optimization, whereas the released model is a single aggregate that generalization itself pushes away from any one training point. This asymmetry sets our design goal. A protocol that eliminates the training-time surface — no plaintext gradient, update, or summary ever observable, and only one round in which anything is exchanged at all — reduces the adversary to the weak inference-time position. That residual position is not a defect any protocol could remove: parties who set out to build a common model accept, by the nature of the undertaking, that the model they jointly release encodes something of each of them. What can be asked of a protocol is that this be the *only* exposure, and we measure it directly (section IV-G).

B. Protecting Training Cryptographically

The cryptographic line protects training-time artifacts without altering the model that is released, in contrast to differential privacy, which perturbs the artifacts themselves and pays for the guarantee in accuracy. Secure aggregation lets the server learn only the sum of the clients' updates, using masks that cancel in the aggregate [34]; it is lossless, but it protects one round's sum inside an iterative protocol, and the revealed per-round sums have themselves been shown to leak across rounds [35], [36]. Homomorphic encryption goes further by computing on contributions that stay encrypted. In its most ambitious form it trains a network entirely under encryption: POSEIDON runs encrypted stochastic gradient descent under multiparty CKKS, with activations replaced by polynomial approximations so they can be evaluated homomorphically [21]. The guarantee is strong and adds no noise, but the approach is iterative and pays for encrypted nonlinearities with a deep multiplicative circuit, repeated bootstrapping, and hours of wall-clock; the delicacy of polynomial activations is documented by a substantial line of work [37], [38], [39], [40], [41]. A lighter line encrypts only the aggregation of an otherwise plaintext training loop: BatchCrypt, FedML-HE, and FedSHE develop packed and quantized encrypted summation for cross-silo training [42], [43], [44], SHE-LoRA selectively encrypts a sensitivity-chosen subset of a low-rank adapter each round [45], and hybrid multi-key schemes reduce a round of secure summation to one client-to-server transmission [46]. All of these remain bound to the iterative protocol: the encryption cost is paid on every round, and what is protected is a per-round sum rather than a complete, one-shot contribution.

Closest to our object, transfer learning itself has been placed under encryption. HETAL trains a softmax classifier head on frozen features entirely under CKKS [47], following a longer line that runs linear and logistic training under homomorphic encryption [48], [49], [50]; a federated variant encrypts the fine-tuned last layer across many rounds [51], a concurrent scheme averages encrypted feature tokens under a single decryption key [52], and recent schemes run low-rank-adapter fine-tuning itself under homomorphic encryption between a model owner and data owners [53], [54]. These works share our structural niche — a frozen public backbone with a small trainable unit — but they run an *optimizer on ciphertexts*: multiplicative depth grows with the step count, softmax and sigmoid must be polynomial-approximated, and most are single-party, one data owner outsourcing computation, rather than a collaboration of mutually distrustful parties. They solve a different problem than ours — outsourcing training for a party that cannot train locally, where our parties can train locally but must not reveal what they trained — and we therefore do not compete with them on encrypted-training efficiency; we contrast the two cost profiles briefly in section IV-F. In our protocol training stays in plaintext on each client's own data, and the only encrypted computation is a single depth-one weighted sum of the final displacements.

C. One-Shot Federated Learning and Lossy Privacy

A parallel line collapses federated learning's many rounds into a single upload and download [55], [56]. Early knowledge-transfer methods had clients exchange predictions on a shared public set [57], [58], in practice repeating the exchange; ensemble distillation trains a server model against aggregated client predictions over many rounds [59]. Genuinely single-round methods arrived with data-free distillation, in which the server distils the clients' models through a generator or synthesized inputs, as in DENSE and its successors [60], [61], and with fusion methods that stitch client models together [62] or aggregate them through a layer-wise posterior [63], [64]. These establish that one round can produce a usable joint model, but they operate in plaintext: the single contribution each client sends — an entire model or a synthetic distillate of its data — is exposed, and by the evidence of section II-A that is precisely the artifact worth attacking.

The one-shot methods that do address privacy use differential privacy. FedAUXfdp releases client contributions under a differentially private mechanism after distillation through a pretrained extractor [65]; related approaches protect a diffusion-generated [66] or teacher-ensemble [67] surrogate, or add noise-free differential privacy to the distillation itself [68]. These are our natural quantitative peers, but their privacy is lossy: the noise a meaningful budget requires is added to the very artifact that is released, so accuracy falls as the budget tightens. The guarantee also differs in kind from a cryptographic one. Differential privacy proves a statistical bound on how much the released artifact can reveal, and buys the bound with accuracy; encryption makes the contribution computationally indistinguishable from random to every party without the key, and costs the model nothing. The same trade-off structures differentially private transfer learning, where DP-SGD on frozen pretrained features reaches strong accuracy at moderate budgets [69], [70], [71], [72], extended to federated low-rank adaptation [73], [74]. These methods share our backbone-plus-adapter structure and confirm that frozen public features are the right substrate for private learning, but they pay a budget-dependent accuracy tax on the released model, and they are centralized or multi-round. Our protocol occupies the same niche with a cryptographic guarantee instead: contributions are protected without perturbation, and the released model carries no privacy-induced noise.

D. Federated Fine-Tuning and Task Arithmetic

A recent line federates parameter-efficient fine-tuning, communicating a low-rank adapter rather than a model. FedIT averages the adapters round by round [75], but per-factor averaging is biased: a client learns $\Delta W = BA$, and averaging factors separately yields $(\sum_j B_j)(\sum_j A_j) \neq \sum_j B_j A_j$, a discrepancy FLoRA removes by stacking factors [76], FlexLoRA by reconstructing full products and re-factoring by SVD [77], HetLoRA by reconciling heterogeneous ranks [78], and FedSA-LoRA by sharing only one factor [79]. The fix we adopt is FFA-LoRA, which freezes the randomly initialized down-projection and trains only the up-projection, so that averaging the trained factor is exact; it was introduced to stabilize multi-round differentially private tuning [23]. Under encryption the choice carries more weight than in plaintext: the bilinearity is not a bias to correct but a cost cliff, since resolving it on the server would take ciphertext-by-ciphertext products, whereas the frozen factor keeps the one-shot encrypted aggregate exact at multiplicative depth one. Algebraically our aggregation is task-vector merging [80], shown equivalent to one-shot federated averaging [81]; that one round of fine-tuning suffices for foundation models is itself established in plaintext [82]. What none of this line provides is any protection of the contributions, which is our subject.

Positioning. Taken together, the four lines place every representative method on three axes — whether the protocol is one-shot, what protects the contributions, and how many rounds pay a cryptographic or communication cost — and locate the gap this paper fills. The multi-round lines either protect nothing (FedDF and FuseFL run tens of rounds in plaintext [59], [62]) or protect each round at recurring cost (POSEIDON's encrypted training [21]; the encrypted aggregation of BatchCrypt, FedML-HE, FedSHE, and SHE-LoRA [42], [43], [44], [45]; secure-aggregation sums [34], [46]). The one-shot line communicates once but sends its single contribution unprotected (DENSE, FedLPA, FedSD2C [60], [63], [64]) or noised, with accuracy falling as the budget tightens (FedKT, FedAUXfdp, FedDiff [67], [65], [66]). The encrypted transfer-learning line shares our frozen backbone but runs its optimizer on ciphertexts and serves a single data owner [47], [51]. A single prior scheme is at once one-shot, federated, and encrypted, but only for a single-layer closed-form learner that cannot adapt a deep representation [22]; none brings that combination — protection that, unlike differential privacy,

leaves the released model unperturbed, under a threshold key no single party can open — to the deep, foundation-scale fine-tuning deployment now requires. That intersection — the training-time surface removed in one round, at multiplicative depth one, with the accuracy of plaintext fine-tuning — is the position HE-OFT occupies, and section IV-H quantifies it at the published setups of the closest one-shot peers.

A further axis is unique to HE-OFT: the prior methods that yield a usable model deliver it to the participants in the clear — the plaintext and differentially private one-shot schemes release it directly, while the encrypted-training line keeps a model encrypted only at the recurring cost of homomorphic nonlinearity across many rounds. HE-OFT can instead withhold the model altogether, keeping the one-shot aggregate encrypted and querying it at multiplicative depth one (section III-H), so the model itself stays confidential and a participant obtains only labels; serving a one-shot, depth-one aggregate that is never released is, to our knowledge, new to federated learning.

We stress that the ingredients are not modular, a federated fine-tuning method with encryption added on top: each design choice in section III — the frozen backbone, the frozen down-projection, the bounded trajectory, the linear rule, the client-side vote — is forced by the constraint that the server computes on ciphertexts.

III. Method

A. Overview

We consider N clients that hold private labelled data over a common label space of C classes and want a shared classifier without pooling their data. Client j holds $\mathcal{D}_j$ of size n_j; the data are heterogeneous, with each client's class proportions drawn from a Dirichlet distribution of concentration α (smaller α means more skew). Two constraints separate our setting from ordinary federated learning. The protocol is *one-shot*: one upload and one download per client, with no iterative exchange. And privacy is *cryptographic*: a client's contribution is never revealed in the clear, and we are unwilling to accept the accuracy loss that differential privacy incurs when it is the only line of defence.

These two constraints, together with the untrusted setting, do not merely shape the design; they determine it. The protocol is not an assembly of independent techniques but the forced solution to a small set of constraints that secure collaborative training imposes, and we arrived at it by discarding alternatives that violate one or another — a sequential or split-training scheme, for example, hands each party the intermediate state its predecessors trained, exactly the exposure **C5** below forbids. The constraints, and the choice each forces, are:

- **C1** *Sophisticated learning cannot run under encryption.* Nonlinear optimisation — gradient steps, softmax, adaptive curvature — needs deep, repeatedly bootstrapped homomorphic circuits. *So* the learning happens in plaintext on the clients, which pre-compute everything the aggregation will consume: the local fine-tune, the per-coordinate importance estimate, and the single displacement each client chooses to upload. The server performs no learning.
- **C2** *The server cannot be adaptive.* Blind to the values it holds, it cannot inspect, compare, or branch, and so cannot pick a data-dependent aggregation rule. *So* it emits several candidate aggregates and defers the choice to the clients, who select among them after decryption (section III-G).
- **C3** *The parties must share a frame before they begin.* In the open-ended landscape of deep-network parameters, independently trained models occupy incomparable regions and their linear combination is meaningless. *So* the parties fix a common geometry in advance: a frozen public pretrained backbone fixes the representation, and a common public initialisation places every client at one point (section III-D).
- **C4** *The server's operation must be shallow.* Homomorphic cost is dominated by multiplicative depth. *So* the one server operation is a depth-one linear combine, and the design is arranged so this cheapest operation is *exactly* the one the method needs — freezing the adapter's down-projection makes the linear sum of the trained factors equal the sum of the weight updates (section III-F).
- **C5** *Intermediate training signals are the strongest attack surface.* Each exposed round is a fresh training-time artifact, far more revealing than the released model (section II-A). *So* the protocol is one-shot: a single encrypted exchange, one artifact, never in the clear.
- **C6** *No single party may be trusted to decrypt.* The parties are mutually distrustful and the server is untrusted; a single decryption key would let its holder reveal every contribution. *So* the key is generated and held distributively — multiparty CKKS with threshold decryption, under which no party, the server included, can decrypt alone (section III-C).

The six form a chain rather than a list: **C1** pushes the work onto the clients, which raises **C2** (a blind server) and **C3** (their returns must be comparable); **C4** and **C6** shape the single operation the server may perform; **C5** fixes how many times it happens. The protocol below is one instance of their joint solution, and each constraint can be relaxed only at a stated price: abandon one-shot encryption and the intermediate snapshots leak (**C5**); abandon encryption altogether and the guarantee weakens from cryptographic to statistical — the differentially private route whose accuracy cost section IV measures. Fine-tuning a pretrained model is thus not a convenience but the learning setting these constraints admit, and it removes the alignment phase that most distinguishes HE-OFT from prior one-shot federated learning, which spends an extra communication round, and often a differential-privacy budget, to bring clients into a common frame before they begin.

Concretely, every client uses a frozen public backbone φ followed by a small trainable unit: low-rank adapters together with a classifier head, with parameters ψ. A low-rank adapter writes the update to a weight matrix as a product $\Delta W = BA$ of two small factors; we freeze the down-projection A at its shared public initialisation and train only the up-projection B and the head. This single choice is what makes the encrypted aggregation *exact*: with A fixed and common to all clients, a

weighted sum of the trained factors reconstructs the weighted sum of the updates, $\big(\sum_j w_j B_j\big)A = \sum_j w_j B_j A$, whereas training both factors would make the update bilinear and the per-factor average a different, and worse, quantity. The backbone never changes, is never transmitted, and is never encrypted; only the trainable unit is fine-tuned, communicated, and operated on under encryption. Keeping that unit small is what makes the encrypted step inexpensive: it is a tiny fraction of the backbone (a low-rank adapter and head, on the order of 10^5 parameters against a backbone of 10^8), so a client's entire encrypted contribution is a few tens of ciphertexts rather than the model itself. Every client initializes the trainable unit to the *same* public constant θ_0, the standard adapter initialization, agreed in advance and carrying no client data. There is no prototype exchange and no peer-to-peer phase; nothing data-derived ever leaves a client before encryption.

The protocol is two steps, shown in fig. 1. Starting from the shared initialization θ_0, each client fine-tunes its trainable unit on its own data for a bounded number of steps and records the displacement it travelled, $\Delta_j = \theta_j^{(K)} - \theta_0$. The server then combines the encrypted displacements into a single global model, $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$, using only homomorphic additions and multiplications by public scalars, after which a threshold of clients jointly decrypts the result.

The protocol consists of four steps, where $\mathsf{Enc}(\cdot)$ denotes encryption under the clients' collective CKKS key. The learning is performed entirely on the clients, and the server's only operation is the single encrypted sum of the third step.

Setup. The N clients run a distributed key-generation protocol that yields a collective CKKS public key, under which any party may encrypt and compute but no party can decrypt alone, decryption requiring a threshold of clients (section III-C).

Local fine-tuning, performed in parallel by each client j. Beginning from $\theta_j^{(0)} = \theta_0$, client j takes K steps of supervised fine-tuning on its own data,

$$\theta_j^{(k)} = \theta_j^{(k-1)} - \eta\, \nabla \mathcal{L}\big(f_\theta;\, \mathcal{D}_j\big), \qquad k = 1, \dots, K,$$

and uploads the encryption $\mathsf{Enc}(\Delta_j)$ of its displacement $\Delta_j = \theta_j^{(K)} - \theta_0$.

Encrypted aggregation, performed by the server. The server forms

$$\mathsf{Enc}(\theta^\star) = \theta_0 + \sum_{j=1}^{N} w_j\, \mathsf{Enc}(\Delta_j),$$

using only multiplications of a ciphertext by the public scalar w_j and additions of ciphertexts, with the public constant θ_0 added in the clear; the circuit has multiplicative depth one. The server broadcasts $\mathsf{Enc}(\theta^\star)$.

Threshold decryption. A threshold of clients jointly decrypts $\mathsf{Enc}(\theta^\star)$ by collective key-switching to recover the global model $\theta^\star$. The server never obtains a plaintext.

The objective is not a model that surpasses each client on that client's own dominant classes; a local specialist on a narrow slice of the label space is difficult to surpass on that slice, and we do not claim to. The objective is a common model every party can use, that classifies well across the whole label space including classes a given client never observed locally, and that is produced without any party exposing its data or its update in the clear.

Algorithm 1 HE-OFT: one-shot federated fine-tuning under multiparty CKKS, with the two disclosure settings.

Public, shared, data-free: frozen backbone φ; initializer θ_0 of the trainable unit (freeze-A adapter + head)
clients run distributed key generation → collective CKKS key ▷ secret-shared; no party decrypts alone
for client $j = 1, \dots, N$ **in parallel do**
  $\theta_j \leftarrow \textsc{FineTune}(\theta_0, \mathcal{D}_j, K)$ ▷ plaintext, on the client
  send $\mathsf{Enc}(\Delta_j)$ to the server, $\Delta_j = \theta_j - \theta_0$
end for
server forms depth-one candidates $\{\mathsf{Enc}(\theta^{(m)})\}$ from $\{\mathsf{Enc}(\Delta_j)\}$: scaling $\theta_0 + \lambda \sum_j w_j \Delta_j$, count-/Fisher-reweighted, leave-one-out ▷ + and public-scalar × only
clients threshold-decrypt the candidates and select $\theta^\star \leftarrow \theta^{(m^\star)}$, $m^\star = \arg\max_m \overline{\mathrm{acc}}_m$ on local holdouts

Disclosure of $\theta^\star$ (section III-H):
Release: threshold-decrypt $\theta^\star$ → every client holds the plaintext model
Serve: keep $\mathsf{Enc}(\theta^\star)$; on query x: $\mathsf{Enc}(\ell) \leftarrow \mathsf{Enc}(\theta^\star)\, \varphi(x)$ (depth 1); $\hat{y} \leftarrow \arg\max_c \ell_c$ under encryption; threshold-decrypt $\hat{y}$ only

TABLE I
NOTATION.

Symbol	Meaning
N	number of participating clients
$\mathcal{D}_j,\ n_j$	client j's local dataset and its size $n_j = \lvert\mathcal{D}_j\rvert$
C	number of classes
α	Dirichlet concentration (label heterogeneity)
φ	frozen public backbone (never trained or encrypted)
ψ	parameters of the trainable unit (adapter + head)
f_θ	model on the frozen features, parameters θ
θ_0	shared public initialization of the trainable unit
K	local fine-tuning steps
Δ_j	displacement $\theta_j^{(K)} - \theta_0$
w_j	sample weight $n_j / \sum_i n_i$
$\theta^\star$	aggregated global model $\theta_0 + \sum_j w_j \Delta_j$

B. Notation

Table I collects the symbols used throughout. Client indices are $j \in \{1, \dots, N\}$.

C. Multiparty Homomorphic Encryption

We build on the CKKS scheme [83], a ring-learning-with-errors construction [84] that performs approximate arithmetic on vectors of real numbers under encryption. CKKS packs many real slots into one ciphertext and supports two homomorphic operations of interest here: addition of two ciphertexts, and multiplication of a ciphertext by a plaintext. It is a levelled scheme, meaning each multiplication consumes one level of a finite budget; our protocol uses only additions and multiplications by public plaintext scalars, so it consumes a single level and never requires the expensive bootstrapping step [85] that refreshes a depleted budget.

A single-key scheme would require one party to hold the secret key and therefore be trusted with decryption. We instead use the multiparty variant of CKKS [86], in which the secret

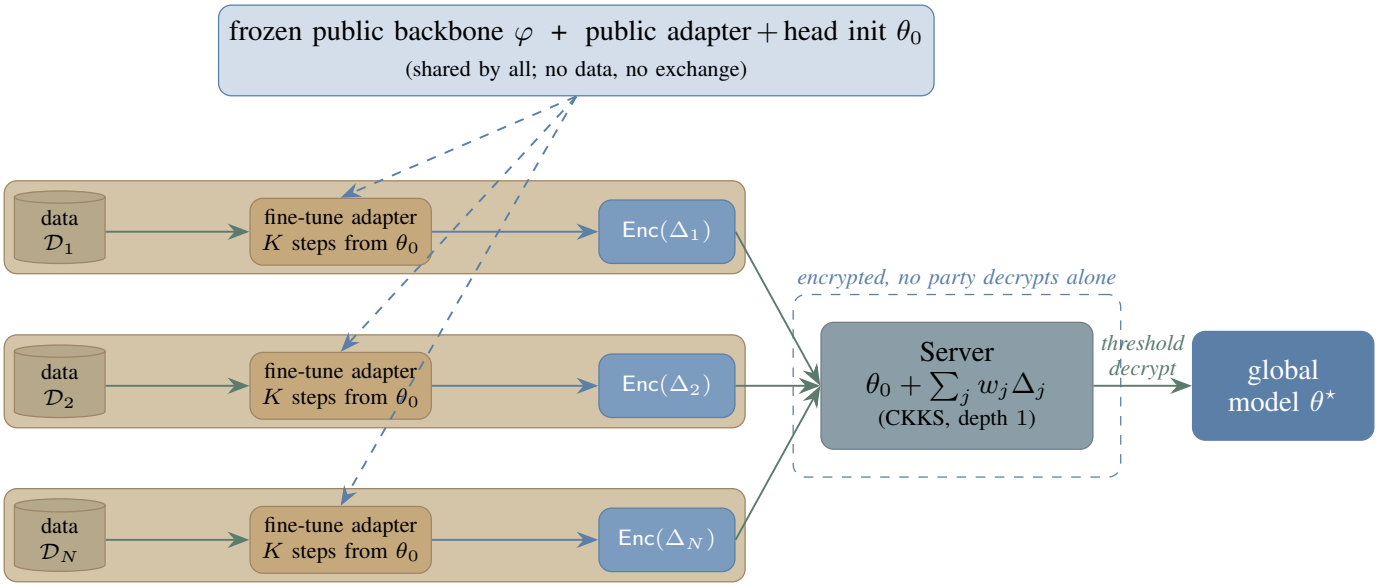

Fig. 1. HE-OFT. A frozen public backbone φ and a public initialization θ_0 of the small trainable unit (low-rank adapter and head) are shared by all clients and carry no data, so there is no alignment phase; the backbone is never trained or encrypted. Each client fine-tunes the adapter on its own private data $\mathcal{D}_j$ for a bounded number of steps from θ_0 and encrypts the resulting displacement $\Delta_j = \theta_j - \theta_0$. The server's sole operation is the sample-weighted linear combination $\theta^\star = \theta_0 + \sum_j w_j \Delta_j$ under multiparty CKKS (additions and public-scalar multiplications only; multiplicative depth one); it never decrypts. A threshold of clients jointly decrypts the global model $\theta^\star$.

key is shared among the clients through a distributed key-generation protocol that produces a collective public key without ever materialising the secret in one place. Any party, including the server, may compute on ciphertexts under the collective public key, but decryption requires the cooperation of a threshold of clients, carried out as a collective key-switching protocol. No single party, and in particular not the server, can decrypt on its own. This is exactly the property our aggregation needs: the server performs the homomorphic computation on encrypted contributions, yet only the clients, acting together, can reveal the result.

We stress what the scheme is chosen for, since our circuit uses almost none of its homomorphic power and a reader may ask why a fully homomorphic scheme is deployed at depth one. The answer is the key structure and the data layout, not the depth. Threshold decryption over a collectively generated key is the property the protocol cannot do without, and the mature distributed key generation and collective key-switching of multiparty CKKS provide it directly [86]. CKKS additionally packs thousands of real-valued parameters into one ciphertext and multiplies them by public scalars natively, the operations the candidate aggregates of section III-G require; additively homomorphic schemes offer neither packed real arithmetic at this width nor comparable multiparty tooling, and masking-based secure aggregation reveals the plaintext sum to the server, which our protocol never does. Depth one is therefore not an underuse of the scheme but the point of the design: the protocol is engineered so that the strongest available tool is needed only at its least expensive setting.

D. The Shared Frame

The natural one-shot recipe (let each client train independently and have the server average the results) fails under heterogeneity. When clients begin from different initializations and fit different skewed distributions, their trained parameters occupy different regions of parameter space and their average lies in none of them: the updates point in conflicting directions and cancel when summed. The remedy is for every client to begin from the same point and move only a little, so their displacements live in one frame and add constructively rather than fighting one another.

A pretrained backbone provides this directly. Because the representation is fixed and shared, the trainable unit lives in the same parameter space for every client, and initializing it to a common public constant θ_0 places all clients at the same point in that space before any training. No data is consulted to build θ_0 and nothing is exchanged to agree on it; it is a fixed, public initializer. A from-scratch federation cannot take this step. It must first agree on a starting point, in a data-dependent alignment phase that costs an extra round and exposes a per-client summary; protecting that summary spends a differential-privacy budget. Fine-tuning a pretrained model removes the phase, and with it the only channel through which client data would otherwise leave in the clear before encryption. We verify in section IV that this shared start is what makes the one-shot combination work (averaging independently trained units without it collapses), and that the frozen backbone supplies the start with no alignment phase.

The frame the backbone supplies is the representation, not the classifier. The shared initializer θ_0 is an untrained head: it places every client at the same point in parameter space but encodes nothing about any class. A client therefore contributes a useful displacement for a class only when it holds examples of that class, so the global model's accuracy on a class is bounded by how many clients cover it. This is the one accuracy cost of dropping alignment, and it is a property of *coverage*

rather than of the frame. It is most visible when the label space is large and the skew extreme, so that each client sees only a sliver of the classes, and it narrows as each client sees more of the label space, closing well before the data become uniform. We characterize this gap directly in section IV, measured against an alignment-based reference that closes it only by exposing per-client class summaries in the clear, the exposure the cryptographic design exists to remove.

E. Local Fine-Tuning

Starting from θ_0, client j fine-tunes its trainable unit on its own data for K steps of ordinary supervised training, producing parameters $\theta_j^{(K)}$. The step budget K is deliberately small. This is not an optimisation shortcut but part of the method: a short trajectory keeps the trainable unit close to θ_0, and therefore inside the shared frame, so the displacement it produces is still expressed in the common coordinates. Allowed to train to convergence, each client would drift to its own local optimum and reintroduce exactly the conflict the shared start was meant to avoid.

At the end of its trajectory the client transmits not its parameters but the displacement

$$\Delta_j \;=\; \theta_j^{(K)} - \theta_0, \tag{1}$$

the distance it travelled from the shared start. Because every client departs from the same θ_0, these displacements are directly comparable across clients, which is what makes the linear combination that follows meaningful.

F. Encrypted Linear Aggregation

The clients hold a joint encryption key produced by distributed key generation, so no single party, and in particular not the server, ever holds the secret key. Each client encrypts its displacement Δ_j; since the trainable unit is small relative to the backbone, this is a few tens of ciphertexts. The server fuses the per-client displacements into a single update of the shared model by the sample-weighted combination

$$\theta^\star \;=\; \theta_0 + \sum_{j=1}^{N} w_j\,\Delta_j, \qquad w_j = \frac{n_j}{\sum_i n_i}, \tag{2}$$

in which each client contributes in proportion to the data n_j behind it. Only the displacements are encrypted; the weights w_j and the initializer θ_0 are public and stay in plaintext. Forming eq. (2) therefore requires only multiplying each ciphertext by the public scalar w_j and adding the results, with θ_0 added in plaintext: there is no ciphertext-by-ciphertext multiplication, no encrypted optimizer state, and no bootstrapping, so the circuit has multiplicative depth one. A threshold of clients then jointly decrypts $\theta^\star$ by collective key-switching, and the server never observes a plaintext update. This is the central design choice: casting aggregation as an operation that is *linear in the encrypted quantities* yields a server computation that maps onto the least costly operations a levelled scheme provides, without the encrypted nonlinearities that make encrypted training expensive. The operations are also fully parallel (the clients fine-tune and encrypt independently, and the server's sum is an associative reduction), so the end-to-end wall-clock is one client's local computation plus a single aggregation, independent of N. We instantiate eq. (2) in a real multiparty CKKS implementation and report its measured correctness and cost in section IV-F.

We state precisely why this works, since eq. (2) can be misread. Since the weights sum to one, the displacement form is algebraically a weighted average of the clients' fine-tuned units, $\theta^\star = \sum_j w_j\,\theta_j^{(K)}$, which is exactly task-vector merging [80]. This linearity is what keeps the operation inexpensive to encrypt, and it is exact rather than approximate only because the adapter's down-projection is frozen: were both adapter factors trained, the per-client update B_jA_j would be bilinear, the average of the factors would not equal the average of the updates, and the quantity the server can compute at depth one would no longer be the one the method requires. The same average over units trained independently, from different initialisations, to convergence is the naive combination that fails under heterogeneity. The average lies in a usable region here because of the protocol around it: every $\theta_j^{(K)}$ starts from the shared θ_0 and moves only a bounded distance, so all of them remain in the common frame and their weighted average is itself a reasonable model. The contribution is not a new aggregation rule but a one-shot protocol under which a linear, and therefore inexpensively encryptable, aggregation produces a strong shared model. One distinction matters throughout: this depth-one linear family *is* task-vector merging, and the plain sample-weighted sum (λ=1) is a single point in it — the "naive" aggregate that heterogeneity destabilises. The client-side selection of section III-G does not swap in a different rule; it picks the point in the same family (a softer λ, or a coverage- or curvature-reweighted merge) that generalises best, so the linearity, and with it the depth-one encryption, holds for every candidate.

G. Multi-Candidate Release and Client-Side Selection

A server computing under encryption is blind: it cannot inspect a value, so it cannot adapt the aggregation rule to the data the way a plaintext aggregator could. We recover that adaptivity without giving the server the ability to read its inputs, by moving the choice to the clients after decryption. The observation we use is that several useful aggregation rules are each, on their own, a depth-one function of the encrypted displacements, so the server evaluates all of them in one pass and the clients select among the results. We form a small set of candidates, in three tiers:

- *Reweighted merges* $\theta_0 + (\sum_j w_j g_j \odot \Delta_j) \oslash (\sum_j w_j g_j)$, where g_j is a per-coordinate weight each client forms from its own data: a diagonal Fisher estimate, or — the variant that carries most of our accuracy under skew — a per-class example count applied to the head rows, so that the clients which actually hold a class decide its classifier row and rows no client covers stay at θ_0. This is the merge the client vote most often selects (section IV-B). Each client forms the products $g_j \odot \Delta_j$ and g_j *before* encryption; the server adds the two ciphertext stacks; and the clients divide the decrypted numerator by

the decrypted denominator in plaintext, so the nonlinear division never occurs under encryption and the server operation stays a depth-one sum.
- a *scaling family* $\theta_0 + \lambda \sum_j w_j \Delta_j$ for a public grid of λ — the plain weighted aggregate (λ=1) and softer steps toward it — which trades the shared start against the aggregated move and costs nothing extra, since all candidates are affine in the same encrypted sum. This is the conservative baseline tier: it discloses nothing beyond the released model (below), and when the data are near-uniform it already suffices; under skew it is the reweighted merges that lift the model above it.
- *leave-one-out* aggregates $\theta_0 + \sum_{i \neq k} \tilde{w}_i \Delta_i$, one per excluded client k, with public renormalised weights $\tilde{w}_i$, used for robustness (section IV-D).

The server emits the encrypted candidates; a threshold of clients jointly decrypts each one; every client scores the decrypted candidates on a small held-out slice of its own data; and the federation releases the candidate with the highest sample-weighted mean held-out accuracy across clients. Selection is therefore post-decryption and entirely client-side: it adds one threshold decryption per candidate and no homomorphic multiplications beyond the depth-one aggregate. It could not usefully move under encryption — it is data-dependent, scored on held-out client data the server does not have, and homomorphic comparison would need deep polynomial approximations of the sign function, exactly the circuit cost the design avoids, for a step the clients perform for free after decryption.

a) What each candidate reveals.: The candidates differ in what they disclose to the decrypting clients, and we account for it plainly. The scaling family is collinear, $\theta^\star(\lambda) = (1-\lambda)\theta_0 + \lambda\theta^\star(1)$, so releasing the whole grid reveals nothing beyond the released model itself. A reweighted merge decrypts its numerator and denominator separately, so it additionally reveals two *aggregate* sums — for the count-weighted head, the federation-wide per-class example totals $\sum_j n_{j,c}$ and the count-weighted head displacement. This is more than the scaling family discloses, but it is consistent with the threat model of section III-I: both are sums over all clients, they isolate no individual client, and the server, which never decrypts, learns nothing from them. A leave-one-out pair, by contrast, does isolate a single client's contribution, and is therefore admitted only when its robustness use (section IV-D) is wanted. Two knobs follow. A deployment content to disclose nothing beyond the released model restricts the set to the scaling family, forgoing the accuracy the reweighted merges would have bought; a deployment that wants the merges but not even the aggregate counts computes the head ratio through a lightweight post-decryption secure division — one secure reciprocal per class and a rescale of each head row, negligible beside the homomorphic aggregation — so that only the final head rows, and neither sum, are ever revealed. Both prices are small, and neither touches the server's depth-one operation.

H. Releasing versus Serving the Aggregate

The protocol so far produces a single object, the selected aggregate $\theta^\star$, held under encryption. One step remains — making it usable — and it admits two settings that trade disclosure against cost. The results of this paper concern the first; the second is available where a deployment may not expose the model at all.

Release. A threshold of clients jointly decrypts $\theta^\star$, and every party holds the shared model in the clear. This is the natural output of a federation whose purpose is a common model, and it is the setting the accuracy, robustness, and membership-inference results below assume. Its only residual surface is the released model itself (sections III-J and IV-G).

Serve. The aggregate is never decrypted. It stays in ciphertext and answers queries under encryption, so that no party — client or server — ever holds the model in the clear, and a client learns only the labels it requests. This fits deployments in which the fine-tuned model is itself a regulated or proprietary asset that may not be released even to its builders. It is affordable for the same reason the aggregation is: the trainable unit is *linear in the frozen features*. For a query x the public backbone runs in plaintext to features $\varphi(x)$, and the encrypted head produces encrypted logits by the very operation the aggregation uses,

$$\mathsf{Enc}(\ell) \;=\; \mathsf{Enc}(\theta^\star)\,\varphi(x), \qquad \hat{y} \;=\; \arg\max_{c \in \{1,\dots,C\}} \ell_c, \quad (3)$$

a plaintext vector against ciphertext weights at multiplicative depth one, so no nonlinearity ever meets the encrypted head. The $\arg\max$ is taken *under encryption* by a homomorphic comparison circuit [87], and a threshold of clients decrypts the predicted label $\hat{y}$ alone.

a) Why the label and not the logits.: Returning logits would defeat the purpose. The head is a linear map on features the client already holds, so a client that reads logits for feature vectors of its choosing recovers the head exactly, by solving a linear system in as many queries as the feature dimension [89]. Releasing only the $\arg\max$ denies this: it reduces an adversary to label-only boundary search [90], [91], which recovers decision boundaries but never the head's coordinates, and at far greater query cost. Where honest use is intrinsically high-volume, so that no per-client budget separates it from extraction, the returned labels can additionally carry calibrated noise — report-noisy-max [92], folded into the smudging the threshold decryption already performs so that no sub-threshold coalition can remove it — yielding a differential-privacy bound on cumulative leakage. This is a query-budget bound, not a cryptographic one: the aggregate remains recoverable in principle, only expensively.

b) Cost, and who serves.: Serving carries one price the rest of the protocol avoids: the encrypted $\arg\max$ is a homomorphic comparison circuit [87], the one operation in the method that is not depth-one, and it is paid per query rather than once. Evaluated naively, as a sequential fold of $C-1$ pairwise comparisons, it costs on the order of minutes for the larger label spaces — roughly twenty minutes at C=77 in our CPU implementation. This is a loose upper bound, not a floor: the $\arg\max$ is ordinary homomorphic computation, and the standard optimisation applies. Packing the C logits into one ciphertext and reducing them with a log-depth SIMD tournament cuts the sequential comparisons from

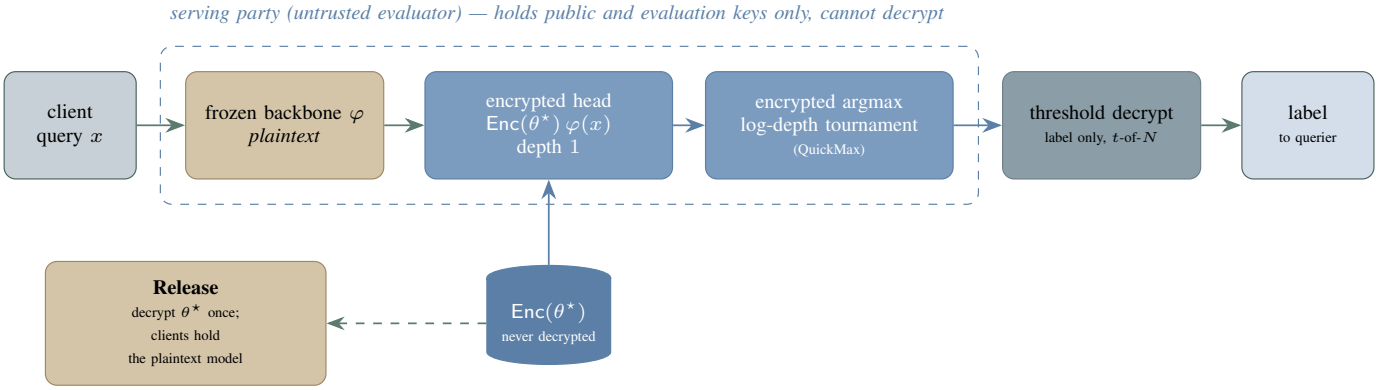

Fig. 2. The two disclosure settings of HE-OFT, sharing the encrypted aggregate $\mathsf{Enc}(\theta^\star)$ produced in fig. 1. *Release* threshold-decrypts $\theta^\star$ once and hands every client the plaintext model. *Serve* never decrypts it: an untrusted serving party — holding only the public and evaluation keys — answers a query x by running the frozen backbone in plaintext, mapping the features through the encrypted head at multiplicative depth one, and taking the $\arg\max$ under encryption; a threshold of clients then decrypts only the predicted label. The model never appears in the clear and a querier learns only labels. The encrypted $\arg\max$ is the one operation above depth one; a log-depth SIMD tournament (QuickMax [88], over homomorphic comparison [87]) evaluates it in $\lceil \log_2 C \rceil$ rounds, and because homomorphic evaluation is identical under a collectively generated key, these single-key costs transfer directly.

$C-1$ to $\lceil \log_2 C \rceil$; our multiparty implementation measures this at under two minutes for a hundred classes, a fourteenfold reduction and exact, and single-key GPU systems built on the same reduction report a full-vocabulary $\arg\max$ in well under a second [88]. None of this is specific to the threshold setting: homomorphic evaluation under a collectively generated key is identical to the single-key case, so these results carry over directly, the multiparty scheme differing only in its distributed key generation and its threshold decryption of the final label. The per-query cost is therefore minutes at worst and sub-second at best, which is why Serve suits low-throughput or governance deployments and Release the rest. The party that answers queries is not thereby trusted with anything: holding only the public key, the evaluation keys, and the ciphertext, it computes but cannot decrypt, and so learns neither the model nor any client's data. It is trusted only for availability and, where honest computation must be certified, through verifiable homomorphic encryption [93]; the decryption threshold is unchanged, since the aggregate becomes plaintext only to a quorum, exactly as under Release.

I. Threat Model

The participants are N clients and a single aggregation server. The server is honest-but-curious: it follows the protocol but may try to infer private information from anything it observes. During aggregation it sees only ciphertexts under the collective key, and as established in section III-C it cannot decrypt them. A minority of clients may likewise be honest-but-curious and may collude, but any coalition below the decryption threshold learns nothing about another client's displacement, since recovering a plaintext requires the cooperation of the threshold. This is a stronger position than secure-aggregation protocols, where the revealed sum has itself been shown to leak across rounds [35], [36], because in our one-shot protocol the server never obtains a plaintext sum at all.

We are explicit about one boundary of this model, which is the training-time/inference-time split of section II-A. The protocol removes the training-time surface entirely: no gradient, update, or data-derived summary is ever observable in plaintext, in any round, because there is only one round and its single artifact is encrypted. Every participant does obtain the released model $\theta^\star$ in the clear after threshold decryption, by design, since the purpose of the protocol is to hand the parties a shared model. Any inference a participant then performs on $\theta^\star$ against another participant's data is therefore not an attack the protocol can prevent but a property of releasing a useful shared model at all, and we measure it directly in section IV-G under exactly this fellow-participant adversary. Where a deployment may not expose even the released model, the Serve setting of section III-H withholds $\theta^\star$ altogether, leaving a participant only the labels of its own queries, at the per-query serving cost discussed there; the contribution-level guarantee stated next is identical in both settings. The cryptographic guarantee concerns the *contributions*: the server and any sub-threshold coalition observe only ciphertexts. This is the sense in which the protection target differs from differentially private one-shot methods, which perturb the released artifact to protect it against all parties, including participants, at a cost to its accuracy. Robustness to actively malicious or Byzantine clients is outside the cryptographic guarantee; the multi-candidate mechanism of section III-G provides a lightweight measured defense (section IV-D), and stronger guarantees we discuss as future work in section IV-J.

J. Security

The model updates are protected cryptographically: the aggregation of eq. (2) runs entirely under multiparty CKKS with threshold decryption, so the displacements Δ_j are never exposed to the server or to any minority of clients, and they are protected without any perturbation. The cryptographic layer adds no accuracy cost. This is the central privacy claim, and the point on which the method departs from differentially private one-shot federated learning, which injects noise into the updates it ultimately releases and so pays a utility tax on the very thing it ships.

We make this precise. Let the view of the honest-but-curious server be everything it receives: the public collective key, the public weights $\{w_j\}$ and initializer θ_0, the client ciphertexts $\{\mathsf{Enc}(\Delta_j)\}_{j=1}^{N}$, and the homomorphically computed $\mathsf{Enc}(\theta^\star)$. We write $\mathsf{view}_{\mathrm{srv}}(\{\mathcal{D}_j\})$ for this view as a function of the private datasets.

Proposition 1. *Under the IND-CPA security of the multiparty CKKS scheme and its t-out-of-N threshold decryption, for any honest-but-curious server colluding with up to $t-1$ clients there is a probabilistic polynomial-time simulator $\mathcal{S}$, given only the public parameters, such that $\mathsf{view}_{\mathrm{srv}}(\{\mathcal{D}_j\}) \overset{c}{\approx} \mathcal{S}$. In particular the server's view is computationally independent of the private datasets $\{\mathcal{D}_j\}$ and the displacements $\{\Delta_j\}$, and the server cannot recover any plaintext, including $\theta^\star$, without a decryption quorum.*

Proof sketch. The simulator outputs the public parameters together with N encryptions of zero under the collective key. By IND-CPA each genuine $\mathsf{Enc}(\Delta_j)$ is computationally indistinguishable from $\mathsf{Enc}(0)$; a hybrid over the N ciphertexts replaces them one at a time, so the whole collection is indistinguishable from the simulated one and carries no information about $\{\Delta_j\}$ or the data they derive from. The aggregate $\mathsf{Enc}(\theta^\star)$ is a deterministic function of those ciphertexts and the public $\{w_j\}, \theta_0$, so it adds nothing. Decryption needs t shares of a secret key that is never reconstructed in one place; the server with at most $t-1$ clients holds too few shares to decrypt. □

Proposition 1 pins down where information can flow. Because there is no alignment phase, no data-derived quantity leaves a client before encryption, so the protocol exposes private information through exactly one channel, intrinsic to the task rather than an artifact of our construction: the released model itself. After threshold decryption every client holds $\theta^\star$ in the clear, and white-box access to the jointly built model is unavoidable in any protocol whose purpose is to hand the parties a shared model. The residual information that model carries about training data is what we quantify with membership inference in section IV-G.

IV. Experiments

We evaluate the protocol's central claim: that encryption protects each client's contribution at no cost to the shared model's accuracy, where one-shot differentially private federated learning must trade accuracy for the same protection. The evaluation tests that claim and the design choices behind it. We ask, in order: what fine-tuning the pretrained layers and selecting among depth-one candidate aggregates add over the naive one-shot average; how the method behaves under heterogeneity, a poisoned client, and the trajectory budget; what the cryptographic protocol costs in time and communication; how much the one remaining surface, the released model, leaks; and what the method reaches at the published setups of prior one-shot methods and on a half-billion-parameter backbone. Throughout, the question is not whether the global model attains the highest possible test accuracy, but whether the clients jointly obtain a single model close to a centrally fine-tuned one, in one round, with cryptographic privacy that adds no accuracy cost.

TABLE II
Evaluation axes. Each varies one dimension around the default ($N = 10$, rank-8 adapter, $K = 200$, three seeds); no public data is used anywhere.

Axis	Setting	Question it probes
Trainable unit	head-only vs. rank-$\{4, 8, 16\}$ adapter	value of fine-tuning pretrained layers
Heterogeneity α	0.05 (severe) to 1.0 (mild)	robustness to label skew
Clients N	10, 20, 50, 100	scalability in participants
Trajectory K	100, 200, 400	the bounded-step budget

A. Setup

Every client starts from the *same* public pretrained backbone with a zero-initialized low-rank adapter and a freshly initialized head — the standard adapter initialization θ_0, which carries no client data and classifies at chance — fine-tunes the adapter and head on its own private data for a bounded K steps, and uploads the encrypted displacement; the server forms the depth-one weighted sum of eq. (2). No labelled public set is used anywhere in the paper, since the setting without one is where a one-shot encrypted aggregation is actually needed (section IV-E discusses the easier case in which a deployment has one). We evaluate on four text classification tasks of increasing label-space size — AG-News (4 classes), TREC (6), DBpedia (14), and Banking77 (77) — with a frozen RoBERTa-base encoder, and on CIFAR-100 with a frozen ViT. Data are partitioned across clients by a Dirichlet distribution over labels with concentration α, the standard label-skew partition [94], [95]: a smaller α leaves each client a few dominant classes and almost none of the rest. This induces label-distribution skew, one axis of heterogeneity in the standard taxonomy [29]; it holds the feature distribution fixed across clients and so does not model cross-institution domain shift, which we leave to future multi-source evaluation. Unless stated otherwise there are $N = 10$ clients, the adapter has rank 8, the trajectory is $K = 200$ steps, and every number is the mean over three seeds.

Alongside the accuracy of the HE-OFT global model we report two reference points: the *naive aggregate*, the plain sample-weighted average of the clients' fine-tuned units with no candidate selection, and, as the ceiling, the *centralized* model, the same adapter and head fine-tuned on the pooled private data, what one party would obtain holding all of it. The quantities of interest are the lift over the naive aggregate and the gap below the centralized ceiling; the centralized accuracy itself appears in captions rather than repeated in every row. Table II lists the evaluation axes.

B. The Value of Fine-Tuning the Pretrained Layers

The design rests on two choices working together: adapting the backbone's own layers with a small frozen-projection

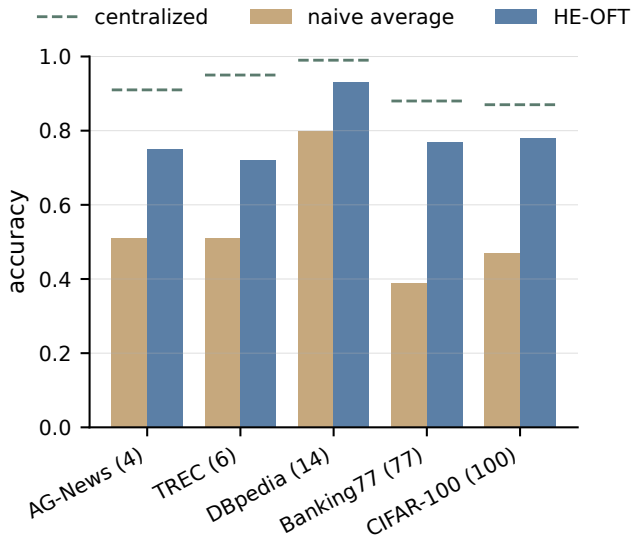

Fig. 3. HE-OFT raises the single encrypted aggregation well above the chance start and above the naive sample-weighted average; the gap to centralized grows with the size of the label space, and no client data is shared.

TABLE III
HE-OFT AGAINST THE NAIVE ONE-SHOT AGGREGATION AND THE CENTRALIZED CEILING ($N = 10$, $\alpha = 0.1$, RANK-8 FROZEN-PROJECTION ADAPTER, THREE SEEDS). "NAIVE" IS THE PLAIN SAMPLE-WEIGHTED AVERAGE (λ=1, NO CANDIDATE SELECTION); HE-OFT IS THE CLIENT-SELECTED CANDIDATE; $\pm$ IS THE SEED STANDARD DEVIATION; THE GAP IS HE-OFT TO CENTRALIZED. TEXT TASKS USE FROZEN ROBERTA, CIFAR-100 A FROZEN VIT-B/16.

Task (C)	naive avg	HE-OFT	centralized	gap
AG-News (4)	0.51	**0.75**$\pm$0.09	0.91	0.16
TREC (6)	0.51	**0.72**$\pm$0.05	0.95	0.23
DBpedia (14)	0.80	**0.93**$\pm$0.01	0.99	0.06
Banking77 (77)	0.39	**0.77**$\pm$0.02	0.88	0.11
CIFAR-100 (100)	0.47	**0.78**$\pm$0.01	0.87	0.09

adapter (small enough to encrypt inexpensively, yet fine-tuning the pretrained representation itself), and selecting among depth-one candidate aggregates after decryption rather than committing to a fixed rule the blind server cannot adapt. Figure 3 and table III measure both at $N = 10$ and moderate heterogeneity, against the naive sample-weighted average and the centralized ceiling.

Two effects combine in table III. First, fine-tuning the frozen backbone's own layers with a small adapter raises the model far above the chance start: from a θ_0 that classifies at chance to between 0.72 and 0.93 across the four text tasks and 0.78 on CIFAR-100, with only the small adapter and head entering the encrypted combination, so the protocol and its cost do not change across tasks. Second, the candidate set with client-side selection (section III-G) raises the released model above the naive sample-weighted average, by 0.24 on AG-News, 0.21 on TREC, 0.13 on DBpedia and 0.38 on Banking77; the naive average is the combination that heterogeneity makes unstable, and the vote selects the candidate that is robust to it. Across these headline cells that winner is the count-weighted head merge in the large majority of runs and the Fisher merge

TABLE IV
TRAINABLE UNIT ON DBPEDIA (N=10, α=0.1, THREE SEEDS): A HEAD ALONE ON THE FROZEN FEATURES VERSUS A FREEZE-A RANK-r ADAPTER PLUS HEAD. "PARAMS" IS THE TRAINABLE COUNT, WHICH SETS THE ENCRYPTED OBJECT'S SIZE; HE-OFT IS THE VOTE-SELECTED ACCURACY; "CENTRALIZED" IS THE SAME UNIT TRAINED ON THE POOLED DATA.

Trainable unit	params	HE-OFT	centralized
head only	11k	0.78$\pm$0.01	0.93
+ rank-4 adapter	84k	**0.93**$\pm$0.01	0.99
+ rank-8 adapter	158k	**0.93**$\pm$0.01	0.99
+ rank-16 adapter	306k	0.91$\pm$0.04	0.99

in the rest, never the plain scaling family: it is the coverage reweighting — the clients that hold a class deciding its head row — not a smaller aggregation step, that recovers what the naive average loses under skew. The remaining gap to the centralized fine-tune is small at moderate label spaces (0.06 on the 14-class DBpedia) and is largest where a single round must cover many classes each client barely observes; even there, freezing the adapter's projection and selecting a coverage-weighted candidate hold the 77-class Banking77 gap to 0.11, down from 0.49 for the naive average. HE-OFT produces the model in one round, with no client exposing its data and with encryption that perturbs nothing.

The lift comes from the adapter, not its rank. Table IV varies the trainable unit on DBpedia: a head alone on the frozen features reaches 0.78, and its own centralized ceiling is only 0.93, because a linear probe cannot adapt the representation. Adding a rank-4 adapter, which fine-tunes the backbone's own layers, lifts the released model to 0.93 and raises the centralized ceiling to 0.99; rank 8 matches it and rank 16 does not improve on either, while tripling the encrypted object. The adapter is therefore what makes fine-tuning worthwhile, and a small rank captures all of it, which is also what keeps the ciphertext count low; we fix rank 8 elsewhere as a mid-point.

C. Sensitivity: Heterogeneity, Client Count, Trajectory Length

How much the released model trails the centralized one is governed by label skew, the axis the single linear aggregation is most exposed to. Table V sweeps the three protocol axes on DBpedia. Under the Dirichlet concentration at $N = 10$, the accuracy degrades gracefully and monotonically, from 0.98 when the data are near-uniform ($\alpha = 1.0$, within 0.01 of centralized) to 0.87 under severe skew ($\alpha = 0.05$), while the centralized fine-tune barely moves. The gap the one-shot aggregation pays is therefore a property of heterogeneity, small when each client sees most of the label space and widening only when each sees a sliver of it; this is the same coverage effect that section III-D characterises, and the candidate selection contains it rather than letting it collapse the model.

The aggregation is also stable as participants are added. Holding $\alpha = 0.1$ and scaling the client count from 10 to 50 (table V, middle), HE-OFT stays at 0.90–0.93 with no downward trend, even as each client's shard thins fivefold; the single linear combination neither destabilises nor washes out as N grows, and the protocol's cost (section IV-F) is the

TABLE V
SENSITIVITY OF HE-OFT ON DBPEDIA ALONG THE THREE PROTOCOL AXES, EACH VARIED AROUND THE DEFAULT ($N = 10$, $\alpha = 0.1$, $K = 200$); CENTRALIZED ≈ 0.99 THROUGHOUT. $\pm$ IS THE STANDARD DEVIATION OVER THREE SEEDS; THE K SWEEP IS A SINGLE SEED.

Heterogeneity α	($N = 10$, $K = 200$)			
	0.05	0.10	0.30	1.00
HE-OFT	0.87±0.05	0.93±0.01	0.96±0.02	0.98±0.00
Clients N	($\alpha = 0.1$, $K = 200$)			
	10	20	50	
HE-OFT	0.93±0.01	0.90±0.02	0.93±0.00	
Trajectory K	($N = 10$, $\alpha = 0.1$)			
	100	200	400	
HE-OFT	0.91	0.93	0.94	

only quantity that scales with it. The trajectory budget behaves the same way (table V, bottom): within the swept range a longer trajectory helps monotonically, from 0.91 at $K = 100$ to 0.94 at $K = 400$, narrowing the gap to centralized to 0.05, the frozen backbone keeping even the longest trajectory's displacements coherent enough to sum.

D. Robustness to a Poisoned Client

The same multi-candidate mechanism that selects the best aggregation rule also gives a measured defense against a misbehaving client, at no homomorphic cost. We add to the candidate set the N leave-one-out aggregates (section III-G) and let one client (the one holding the largest shard, the worst case for sample weighting) upload a crafted displacement instead of an honest one: a sign-flipped update, a large-norm Gaussian, or one trained on label-flipped data. The honest clients vote on their local holdouts over the plain aggregate and the N leave-one-out aggregates; the candidate that excludes the attacker is selected whenever its exclusion improves the held-out accuracy.

Table VI reports the result on DBpedia and AG-News at $N = 10$, three seeds. The undefended plain aggregate is pulled down to 0.25–0.50 depending on the attack, while the vote-selected model recovers to within rounding of the attacker-free oracle, and the vote identifies and excludes the attacker in 17 of 18 cells. The one miss is a Gaussian attack whose corruption was mild enough that including the attacker still scored best on the holdouts, i.e. the vote kept it precisely because it was not harmful. This is not a general Byzantine guarantee (it assumes an honest holdout majority and a single attacker), but it shows the candidate machinery already absorbs a poisoning client that the blind server cannot filter.

E. When Public Data Is Available

We study the harder setting in which no public data exists, because that is where a one-shot encrypted aggregation is actually needed. When a deployment does have a public labelled set, the problem is strictly easier and standard recipes apply directly: the shared initialization can be warm-started on it, or a model can be trained on the public data and then refined. With enough of it, training a model from scratch on the public set is itself an option. We therefore treat public data as outside the method and rely on none of it anywhere in this paper.

TABLE VI
ROBUSTNESS TO ONE POISONED CLIENT ($N = 10$, $\alpha = 0.1$). EACH ACCURACY IS THE MEAN OVER THE TWO TASKS (DBPEDIA, AG-NEWS) AND THREE SEEDS, SIX RUNS PER ATTACK. "PLAIN" IS THE UNDEFENDED SAMPLE-WEIGHTED AGGREGATE THAT INCLUDES THE ATTACKER; "HE-OFT" IS THE VOTE-SELECTED LEAVE-ONE-OUT CANDIDATE; "ORACLE" IS THE ATTACKER-FREE AGGREGATE; "EXCLUDED" COUNTS THE RUNS IN WHICH THE VOTE DROPPED THE ATTACKER.

Attack	plain	HE-OFT	oracle	excluded
sign-flip	0.25	**0.65**	0.65	6/6
large-Gauss	0.50	**0.67**	0.65	5/6
label-flip	0.48	**0.65**	0.65	6/6

F. Homomorphic-Encryption Implementation and Cost

We do not simulate the cryptography. We implement the entire protocol end to end in real multiparty CKKS on the Lattigo library [96], [86]: distributed key generation, encryption under the joint public key, the homomorphic weighted aggregation of eq. (2), and threshold decryption by a collective key-switch, and the decrypted aggregate matches the plaintext computation to within CKKS encoding precision (verified below). The server's only cryptographic operation is that depth-one weighted sum, and its cost is set entirely by the number of *fine-tuned* parameters that enter a ciphertext, not by the frozen backbone behind them. We measure the per-operation rates directly and calculate every reported configuration from them, so a small adapter and head are inexpensive to encrypt whatever model they sit on.

a) Parameters and setup.: All figures use ring degree 2^{14} (8192 slots per ciphertext), a modulus chain $\log Q = \{55, 45\}$ with key-switch prime $\log P = \{61\}$, and scale 2^{45}. The circuit is multiplicative *depth one* (one plaintext-scalar multiplication and a rescale, with no relinearization and no bootstrapping), and decryption is N-out-of-N (no single party can decrypt). Communication figures are exact; timings are single-run wall-clock on one core of a commodity laptop CPU (Apple M4) and are reported as indicative. The circuit uses no ciphertext rotations and no relinearization, and because the only server operation is the one-shot aggregation, there is no encrypted training loop and hence no convergence behaviour to report under encryption. The server accumulates the weighted sum as a streaming reduction, holding at most two adapter-sized ciphertext sets at once (about 38 MiB), so its memory does not grow with N.

b) Communication.: A round is one upload and one download, and the protocol completes in a single round. The per-client traffic is fixed by the trainable-parameter count: a unit of up to 8192 parameters is a single 0.5 MiB ciphertext. The headline configuration, a freeze-A rank-8 adapter with a classifier head (about 1.5×10^5 trainable parameters), occupies 19 ciphertexts, or 9.5 MiB per client; freezing the down-projection is what halves this relative to training both factors. Total traffic scales linearly in the number of clients (table VII). The contrast with encrypted full-model schemes is structural: on the same backbone, encrypting the full RoBERTa-base

TABLE VII
CRYPTOGRAPHIC COST PER TRAINABLE UNIT (MULTIPARTY CKKS, RING 2^{14}, 0.5 MiB PER CIPHERTEXT; TIMES IN MS ON ONE CPU CORE, APPLE M4). THE MiB COLUMN IS THE PER-CLIENT UPLOAD, EQUAL TO THE DOWNLOAD; THE PROTOCOL COMPLETES IN ONE ROUND, AND TOTAL TRAFFIC SCALES LINEARLY IN N (FIG. 4). CLIENTS ENCRYPT IN PARALLEL; AGGREGATION AND THRESHOLD DECRYPTION SCALE LINEARLY IN $N\times$CIPHERTEXTS; THE ONE-TIME DISTRIBUTED KEY GENERATION COSTS ≈ 11 MS AT $N{=}10$ AND ≈ 96 MS AT $N{=}100$.

				aggregate / decrypt (ms)	
Trainable unit	ct.	MiB	enc. (ms)	$N{=}10$	$N{=}100$
freeze-A adapter	19	9.5	40	76 / 44	717 / 430
text head (4)	1	0.5	2.3	3.9 / 2.4	38.9 / 23.2
vision head (100)	10	5.0	21.5	39.0 / 24.6	382.2 / 216.5

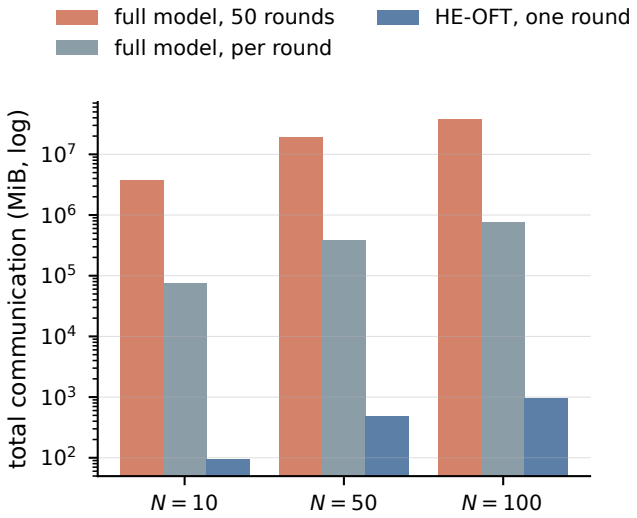

Fig. 4. Total communication on a shared backbone (RoBERTa-base), calculated for CKKS ring 2^{14} (0.5 MiB per ciphertext; 50 rounds for the iterative schemes). HE-OFT encrypts only the adapter and runs once; full-model encrypted schemes move gigabytes per round (BatchCrypt, FedML-HE, FedSHE) to terabytes over training (POSEIDON-style encrypted training).

(1.25×10^8 parameters) would move about 7.5 GiB per client *per round*, and such schemes run for many rounds, where HE-OFT encrypts only the adapter and runs once (fig. 4). The object stays small even on a half-billion-parameter backbone: a freeze-A adapter on the 0.5B Qwen2.5 model is 26 ciphertexts (13 MiB), because only the adapter is encrypted, never the backbone.

c) Computation.: The server's homomorphic work is tens of milliseconds at deployment scale and stays sub-second even at $N = 100$ (table VII). The measured operations follow simple laws: a client encrypts in about 2.1 ms per ciphertext (in parallel across clients), while the server aggregation and the threshold decryption scale linearly in $N\times$(ciphertexts/client), and the one-time distributed key generation grows linearly in N at about 1 ms per party. From these rates the freeze-A adapter (19 ciphertexts) costs about 40 ms to encrypt per client and, at $N = 100$, about 0.72 s to aggregate and 0.43 s to decrypt (table VII). Emitting k candidate aggregates (section III-G) costs k threshold decryptions, so a dozen candidates add about half a second; there is no bootstrapping anywhere in the protocol.

d) Comparison with other encrypted schemes.: Table VIII places HE-OFT against the encrypted federated-learning approaches. Two properties separate it from all of them. First, it is one-shot: it encrypts a single contribution and runs one round, where encrypted-training schemes such as POSEIDON [21] run multiparty CKKS for many rounds, hours of wall-clock even on small models, and aggregation schemes such as BatchCrypt, FedML-HE, and FedSHE [42], [43], [44] encrypt and sum updates on every round of an iterative protocol. Second, it encrypts only the small adapter and head, not the full model: the per-client object is 9.5 MiB against the gigabytes a full-model scheme moves on the same backbone, and there is no bootstrapping because the circuit is depth one. The closest recent scheme, SHE-LoRA [45], also encrypts only adapter parameters, but it is multi-round and pays its encryption cost on every round, and because it trains both adapter factors it carries the bilinear aggregation noise under encryption that our frozen-projection design removes. The secure-aggregation primitive of [46] is one-round but protects a per-round sum within an iterative protocol rather than a one-shot contribution. Figure 4 shows the per-round communication gap on a shared backbone, before the round-count multiplier that further separates a one-round protocol from a multi-round one.

The encrypted object stays depth one and the server cost grows only with the ciphertext count, never with the size of the frozen backbone the adapter sits in.

e) Comparison with encrypted transfer learning.: HETAL [47] solves a neighbouring but different problem: a single data owner outsources training, so the optimizer itself must run on ciphertexts — an encrypted gradient method with approximated softmax and bootstrapping, at a reported 567–3442 s of encrypted training per benchmark on an NVIDIA A40 GPU. Our parties can train locally and must only hide what they trained, so the optimizer never touches a ciphertext, and the entire encrypted pipeline (encrypt, aggregate, threshold-decrypt) is 0.16–1.2 s on one laptop CPU core, exact to a relative $\ell_2 \approx 10^{-9}$. The two designs are complementary rather than competing; the contrast quantifies what moving the encryption boundary from the optimizer to a one-shot aggregation buys.

f) Correctness.: The decrypted aggregate matches the plaintext computation to a relative ℓ_2 error between 2.7×10^{-9} ($N = 10$) and 2.6×10^{-8} ($N = 100$). This is the encoding precision of CKKS at scale 2^{45}; it grows mildly with N as ciphertext additions and the key-switch smudging noise accumulate, and it stays several orders of magnitude below any tolerance the decoded model is sensitive to. Because the circuit is depth one, the error is not amplified.

g) Comparison with prior cryptographic federated learning.: The contrast with prior homomorphic federated learning is structural, not incremental. Encrypted-training methods such as POSEIDON [21] reach comparable accuracy but through many rounds of homomorphic computation that the source paper reports at roughly 1.4 hours of wall-clock; secure-aggregation primitives [46] sum updates once per round inside

TABLE VIII
HE-OFT AGAINST ENCRYPTED FEDERATED-LEARNING SCHEMES. COMMUNICATION IS PER CLIENT; HE-OFT'S IS CALCULATED FOR A FREEZE-A ADAPTER (ROBERTA-BASE, 19 CIPHERTEXTS), THE FULL-MODEL FIGURE FOR ENCRYPTING THE SAME BACKBONE.

Method	cryptographic scheme	encrypted object	rounds	communication per client
HE-OFT (ours)	multiparty CKKS	adapter+head	**1**	9.5 **MiB, once**
SHE-LoRA [45]	CKKS (selective)	adapter subset	many	adapter-scale, every round
POSEIDON	multiparty CKKS	full model	many	GiB, every round
BatchCrypt / FedML-HE / FedSHE	CKKS	full model	many	GiB, every round
Hyb-Agg	multi-key CKKS	update sum	per round	one transmission per sum

an iterative protocol. Our protocol runs in a single round whose server computation is milliseconds and whose communication is a few megabytes, because the only encrypted object is a bounded fine-tuning displacement of a small adapter, combined at multiplicative depth one. The broader method-level positioning closes section II.

G. Residual Leakage: Measuring the Inference-Time Surface

The design goal set in section I was to eliminate the training-time attack surface, and the protocol achieves it by construction: no gradient, update, or data-derived summary is ever observable in plaintext, and there is only one round in which anything is exchanged at all. What the cryptographic guarantee of section III-J does not cover is the final model, which every party obtains in the clear after decryption. This is the *inference-time* surface of section II-A, and it is not a gap a stronger mechanism could close: any protocol that hands separate parties a shared, useful model must move knowledge between them, and a model that classifies categories a client never saw locally necessarily encodes information about the data that taught it those categories. The meaningful question is not whether this residual leakage is zero, which it cannot be, but how large it is, and the evidence of section II-A is that it is the weakest surface an adversary can be left with. This section measures it directly.

Read as attack surfaces, the alternatives differ by orders of magnitude. Multi-round federated learning publishes a fresh update every round, so a curious server or peer accumulates a long sequence of training-time views of each client [30], [14]. One-shot methods that rely on differential privacy ship, in a single object, essentially the whole of a client's local knowledge: a synthetic copy of its data, or a noised model. HE-OFT exposes only the final aggregate: the per-client displacements are encrypted and never appear in the clear, and because there is no alignment phase there is no prototype channel either.

We measure the residual on that one open channel with membership-inference attacks [19], [31], [20], training the target and a population of shadow models through the full federated pipeline and attacking the released model with the likelihood-ratio attack LiRA (table IX). On the text tasks other than Banking77 the leakage is at the level of random guessing: LiRA reaches an AUC of 0.49–0.50 and a true-positive rate of about 1% at a 1% false-positive rate, indistinguishable from chance. On the 77-class Banking77 it is mildly elevated, to an AUC of 0.59 and a 3.4% true-positive rate, the one text task where coverage forces the model to memorise the most per class, and exactly the task where the accuracy gap to centralized is largest. The same coverage effect produces a similarly mild elevation on the 100-class CIFAR-100 — an AUC of 0.62 and a 3.7% true-positive rate — so the two largest label spaces are precisely where the residual appears, and even there it stays modest.

TABLE IX
MEMBERSHIP INFERENCE AGAINST THE RELEASED MODEL: EXTERNAL ADVERSARY, LIRA, THREE SEEDS. AUC AND THE TRUE-POSITIVE RATE AT A 1% FALSE-POSITIVE RATE; 0.50 AUC AND 1% TPR ARE CHANCE. A FELLOW CLIENT IS NO STRONGER UNDER LIRA (ITS CLASS PRIOR CANCELS IN THE LIKELIHOOD RATIO; SEE TEXT), SO THESE EXTERNAL FIGURES ARE THE OPERATIVE ONES.

Task (C)	AUC	TPR@1%
AG-News (4)	0.50	0.9%
TREC (6)	0.49	0.7%
DBpedia (14)	0.50	0.9%
Banking77 (77)	0.59	3.4%
CIFAR-100 (100)	0.62	3.7%

The threat model of section III-I admits a stronger adversary than the external one: a fellow client that holds its own labelled data, and with it a per-class prior an outsider lacks. The two adversaries therefore run *different* attacks — the external one LiRA, the fellow one a class-conditionally standardized loss threshold, the attack its extra class knowledge actually enables — and comparing them isolates whether observing more buys anything. It does not, for two reasons that meet in the middle. First, handing the class prior to LiRA changes nothing: LiRA already calibrates each example against the shadow models that did and did not train on it, a per-example conditioning that subsumes any coarser class-level one, so a per-class location shift cancels in the likelihood ratio and a fellow-run LiRA is numerically identical to the external one (which is why table IX reports the external). Second, the fellow's own class-conditional threshold — the one attack its prior does move — is a weaker instrument than LiRA to begin with, and stays at or below it on every task, weaker even on Banking77 where there is signal to exploit. Observing more therefore yields no additive advantage against the released model: the stronger attack ignores the extra knowledge, and the attack that uses the extra knowledge is the weaker one. Being a participant rather than an outsider does not increase what one can learn about another's membership, beyond what any usable classifier of that data would already reveal.

We do not claim the released model leaks nothing. A model that gives every client coverage of classes it never observed must carry information about the data that supplied that coverage, and an adversary with strong priors about a

TABLE X
HE-OFT AT PRIOR ONE-SHOT METHODS' DATA PARTITIONS, ON OUR OWN FROZEN VIT-B/16 + FREEZE-A ADAPTER (THREE SEEDS) — A STRONGER BACKBONE THAN THE COMPARATORS', SO THE COLUMNS ARE NOT AN EQUAL-CAPACITY HEAD-TO-HEAD. COMPARATOR NUMBERS ARE PAPER-VERBATIM AT THE CITED SETUP.

Matched setup	data, N, α	HE-OFT	reported
DENSE [60]	CIFAR-10, 5, 0.1	$\mathbf{0.96}\pm0.00$	0.50
DENSE [60]	CIFAR-10, 5, 0.3	$\mathbf{0.96}\pm0.00$	0.60
FedAUXfdp [65]	CIFAR-10, 20, 0.04	$\mathbf{0.94}\pm0.01$	0.75‡
FedAUXfdp [65]	CIFAR-10, 20, 0.16	$\mathbf{0.96}\pm0.01$	0.75‡
FedSD2C [64]	Tiny-ImageNet, 10, 0.1	$\mathbf{0.73}\pm0.01$	—
FedSD2C [64]	Tiny-ImageNet, 10, 0.3	$\mathbf{0.73}\pm0.02$	—

‡FedAUXfdp at ε=0.5 on its CIFAR-10 setup.

client's distribution can recover some of it; this is a property of useful shared models, not of our protocol in particular. What the protocol guarantees is that this residual is the *entire* surface (not multiplied across the rounds of an iterative protocol, and not handed over wholesale as released data), and driving the exposed surface down to that inherent floor, rather than promising to remove a leak that cannot be removed when parties set out to build a common model, is the privacy goal the design meets.

H. At Prior Methods' Partitions, and Scale

The positioning paragraph closing section II separates the methods qualitatively; the accuracies each paper reports are obtained at its own setup and do not compare directly. To make the comparison controlled, we run HE-OFT at the *published partitions* of three one-shot methods, matching the dataset, client count, and Dirichlet α but keeping our own model class — a frozen ViT-B/16 with a freeze-A adapter, stronger than the comparators' backbones, which we state plainly. The result is therefore not a head-to-head comparison at equal model capacity; it isolates what the protocol reaches when the substrate is a modern frozen backbone, holding the data partition fixed to each prior method's. Table X reports it. At the data-free DENSE setup (CIFAR-10, $N = 5$) HE-OFT reaches 0.96 against their reported 0.50/0.60; at the differentially private FedAUXfdp setup (CIFAR-10, $N = 20$, severe skew) 0.94 against 0.75 at ε=0.5; and at the FedSD2C setup (Tiny-ImageNet, $N = 10$) 0.73. The gains owe much to the stronger frozen backbone, which is the point: the protocol's cryptographic and communication structure carries unchanged onto a backbone that makes the task easy, where prior data-free and differentially private one-shot methods cannot reach the same accuracy and, in the DP case, pay an explicit budget for a weaker guarantee than encryption gives. The same backbone is also why the CIFAR-10 cells cluster in a narrow 0.94–0.96 band across partitions: the frozen representation, not the partition, sets the ceiling, and the seed spread within each cell stays between 0.002 and 0.02.

The protocol also carries to a half-billion-parameter backbone. On a frozen Qwen2.5-0.5B causal language model with a freeze-A adapter (three seeds), HE-OFT reaches 0.85–0.88 on DBpedia, where the naive average is far lower and swings widely across seeds (0.44–0.63), and 0.71–0.72 on AG-News, while the encrypted object stays at 26 ciphertexts (13 MiB): the cost is set by the adapter, not by the 0.5B backbone behind it, which is never encrypted.

I. Beyond Pretrained Backbones

The protocol does not require a pretrained backbone; it requires a shared frame. Where clients instead train a small network from a common random initialization, the same one-shot encrypted aggregation applies, but the shared frame must be established rather than inherited. Without a pretrained representation the clients' bounded updates are expressed in a coordinate system fixed only by the random seed, so agreeing on a useful starting point reintroduces the kind of alignment step a pretrained backbone removes. We therefore present pretrained fine-tuning as the primary setting and treat the from-scratch variant as a generalization whose alignment cost is left to future work.

J. Scope and Limitations

Two boundaries follow from well-understood properties rather than incidental shortcomings. First, the single linear aggregation pays a gap to centralized fine-tuning that widens with the size of the label space under severe skew, because a client contributes signal only for classes it holds and the shared head is untrained; this is an inherent property of combining bounded one-shot updates without an alignment phase, largest on the 77-class Banking77, and the coverage-weighted candidate contains it (to a gap of 0.11) rather than removing it. Second, the method fine-tunes a pretrained backbone, so it inherits that backbone's fit to the task and applies to the label-groundable foundation models used for transfer today rather than to a backbone with no relevant pretraining. Both are stated where they apply; neither touches the privacy claim, which holds regardless of the accuracy the shared model reaches.

a) Malicious and colluding clients.: Our cryptographic guarantees assume honest-but-curious participants. An actively malicious client could upload a crafted displacement to bias the shared model, and because the contributions are encrypted the server cannot inspect them, so plaintext outlier filtering does not apply. The leave-one-out candidates of section IV-D give a lightweight client-side defense that already neutralises a single poisoning client without any server-side filtering, but it assumes an honest-majority holdout and does not cover colluding or adaptive adversaries. Stronger guarantees compatible with the encryption are left to future work: an upload-time norm bound enforced by a zero-knowledge proof, which caps any single client's influence without revealing its update, and encrypted-domain robust aggregation evaluated homomorphically. The one-shot structure shrinks the attack surface to a single round but removes the cross-round consistency checks a multi-round protocol could use, and reconciling robust aggregation with depth-one encryption is the open problem this direction must solve.

V. CONCLUSION

We presented HE-OFT, a one-shot federated fine-tuning protocol built around a single observation about where federated

learning is vulnerable: the strongest known attacks exploit training-time artifacts, the gradients, updates, and summaries a protocol exposes while the model is being built, and these are far more damaging than anything an adversary can extract from the released model afterwards. HE-OFT removes that training-time surface entirely rather than perturbing it. Each client fine-tunes a small adapter on a frozen pretrained backbone in plaintext and uploads one encrypted displacement, and the server's only operation is a linear aggregation computed under multiparty homomorphic encryption at multiplicative depth one, decrypted jointly by a threshold of clients. Fine-tuning is what makes this affordable: the frozen backbone supplies the shared frame a one-shot linear aggregation needs under heterogeneous data, freezing the adapter's down-projection makes that aggregation exact task arithmetic rather than an approximation, and the encrypted object is a few megabytes whatever the size of the backbone behind it, at no accuracy cost. Because a server computing under encryption cannot adapt its rule to data it cannot read, it emits several depth-one candidate aggregates and the clients select among them after decryption, which improves the released model under heterogeneity and yields a measured defense against a poisoning client, at no homomorphic cost. Across vision and language tasks, from a frozen RoBERTa and ViT to a half-billion-parameter language model, the method yields a common model stronger than any single client trains alone and, outside the most skewed regimes, close to a centrally fine-tuned one; a multiparty CKKS implementation reproduces the encrypted aggregation within the scheme's approximation bounds at about ten megabytes per client; and the one surface the protocol cannot remove, the released model, shows membership inference at or near chance, rising only on the largest label space. Extending the protocol to colluding or adaptive malicious participants, where verifiable homomorphic encryption [93], [97] could certify that the server computed the aggregation honestly, is the natural direction for future work.

ACKNOWLEDGMENTS

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude Sonnet 4.6 and Claude Opus 4.6 to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

REFERENCES

[1] R. Bommasani, D. A. Hudson, E. Adeli *et al.*, "On the opportunities and risks of foundation models," *arXiv preprint arXiv:2108.07258*, 2021.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[3] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 12:1–12:19, 2019.

[4] N. Rieke, J. Hancox, W. Li, F. Milletarì, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein *et al.*, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, p. 119, 2020.

[5] European Parliament and Council of the European Union, "Regulation (eu) 2016/679 (general data protection regulation)," Official Journal of the European Union, L 119, pp. 1–88, 2016, art. 5(1)(b–c) purpose limitation and data minimisation; Art. 9 special categories; Chapter V transfers to third countries.

[6] U.S. Department of Health and Human Services, "HIPAA privacy rule," 45 C.F.R. Parts 160 and 164, 2013, § 164.502(b) minimum necessary; § 164.514(a–b) de-identification (Expert Determination and Safe Harbor); § 164.508 authorization; § 164.502(e) business-associate agreements.

[7] Republic of Türkiye, "Law on the protection of personal data (kişisel verilerin korunması kanunu), law no. 6698," 2016, art. 6 special categories of personal data; Art. 5 processing conditions; Art. 9 transfer abroad (as amended by Law No. 7499, 2024).

[8] Standing Committee of the National People's Congress, "Personal information protection law of the people's republic of china," 2021.

[9] Parliament of India, "Digital personal data protection act," 2023.

[10] European Parliament and Council of the European Union, "Regulation (eu) 2024/1689 (artificial intelligence act)," Official Journal of the European Union, L, 2024/1689, 2024, art. 10 data and data governance for high-risk AI systems; Annex III; Art. 6(1) with Annex I.

[11] G. Xiao, J. Lin, and S. Han, "Offsite-tuning: Transfer learning without full model," *arXiv preprint arXiv:2302.04870*, 2023.

[12] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, vol. 32, 2019.

[13] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients – how easy is it to break privacy in federated learning?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[14] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019, pp. 739–753.

[15] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *2019 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2019.

[16] O. Zari, C. Xu, and G. Neglia, "Efficient passive membership inference attack in federated learning," arXiv:2111.00430; poster, NeurIPS Workshop on Privacy in Machine Learning, 2021.

[17] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, "When the curious abandon honesty: Federated learning is not private," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 175–199.

[18] L. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, "Robbing the fed: Directly obtaining private data in federated learning with modified models," in *International Conference on Learning Representations (ICLR)*, 2022.

[19] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *31st IEEE Computer Security Foundations Symposium (CSF)*, 2018, pp. 268–282. [Online]. Available: https://arxiv.org/abs/1709.01604

[20] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *IEEE S&P*, 2022, pp. 1897–1914. [Online]. Available: https://arxiv.org/abs/2112.03570

[21] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.

[22] O. Fontenla-Romero, B. Guijarro-Berdiñas, E. Hernández-Pereira, and B. Pérez-Sánchez, "FedHEONN: Federated and homomorphically encrypted learning method for one-layer neural networks," *Future Generation Computer Systems*, vol. 149, pp. 200–211, 2023.

[23] Y. Sun, Z. Li, Y. Li, and B. Ding, "Improving LoRA in privacy-preserving federated learning," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2403.12313.

[24] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[25] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023, pp. 12 198–12 207.

[26] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, ser. Lecture Notes in Computer Science, vol. 13423. Springer, 2023, pp. 364–378.

[27] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023. [Online]. Available: https://dl.acm.org/doi/10.1016/j.jisa.2023.103475

[28] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *PoPETs*, vol. 2025, no. 4, pp. 201–215, 2025.

[29] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[30] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE S&P*, 2017, pp. 3–18. [Online]. Available: https://arxiv.org/abs/1610.05820

[31] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *Network and Distributed System Security Symposium (NDSS)*, 2019. [Online]. Available: https://arxiv.org/abs/1806.01246

[32] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015, pp. 1322–1333. [Online]. Available: https://dl.acm.org/doi/10.1145/2810103.2813677

[33] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017, pp. 603–618.

[34] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *ACM CCS*, 2017, pp. 1175–1191.

[35] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, vol. 37, no. 8, 2023, pp. 9925–9933.

[36] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*. ACM, 2023, pp. 15–30.

[37] K. Garimella, N. K. Jha, and B. Reagen, "Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning," in *PPML Workshop, CCS*, 2021.

[38] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkowich, "A methodology for training homomorphic encryption friendly neural networks," in *ACNS Workshops*, ser. Lecture Notes in Computer Science, vol. 13285. Springer, 2022, pp. 536–553.

[39] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E108.A, no. 7, 2025.

[40] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.

[41] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *Privacy in Machine Learning Workshop (PriML), NeurIPS*, 2021. [Online]. Available: https://www.eurecom.fr/en/publication/5626

[42] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning," in *USENIX ATC*. USENIX Association, 2020, pp. 493–506.

[43] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, "Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system," in *NeurIPS*, vol. 36, 2023.

[44] X. Wei, R. Huang, L. Lei, and J. Wu, "Fedshe–cq: A communication-efficient homomorphic encryption framework for federated learning," *Computer Networks*, vol. 260, p. 111813, 2025.

[45] J. Li *et al.*, "SHE-LoRA: Selective homomorphic encryption for federated tuning with heterogeneous LoRA," *arXiv preprint arXiv:2505.21051*, 2025.

[46] Kemmaka and Tran, "Hyb-Agg: Communication-efficient secure aggregation via hybrid multi-key homomorphic encryption and masking," *arXiv:2511.23252*, 2025.

[47] S. Lee, G. Lee, J. W. Kim, J. Shin, and M.-K. Lee, "HETAL: Efficient privacy-preserving transfer learning with homomorphic encryption," in *International Conference on Machine Learning (ICML)*, 2023, arXiv:2403.14111.

[48] M. Kim, Y. Song, S. Wang, Y. Xia, and X. Jiang, "Secure logistic regression based on homomorphic encryption: Design and evaluation," *JMIR Medical Informatics*, vol. 6, no. 2, p. e19, 2018.

[49] A. Kim, Y. Song, M. Kim, K. Lee, and J. H. Cheon, "Logistic regression model training based on the approximate homomorphic encryption," *BMC Medical Genomics*, vol. 11, no. Suppl 4, p. 83, 2018.

[50] J. Chiang, "Privacy-preserving CNN training with transfer learning: Two hidden layers," *arXiv preprint arXiv:2504.12623*, 2025.

[51] A. M. I. M. Osmani, T. Rahman, M. S. Rahman, and S. Islam, "Priv-FedTL: Privacy-preserving federated transfer learning for resource constrained devices," *Computer Networks*, vol. 286, p. 112449, 2026.

[52] M. Al Amin, M. Hasan, S. Hong, and S. Ullah, "Privacy-preserving federated vision transformer learning with lightweight homomorphic encryption in medical AI," *arXiv preprint arXiv:2511.20983*, 2025.

[53] Y. Li, W. Yu, and J. Zhao, "PrivTuner with homomorphic encryption and loRA: A P3EFT scheme for privacy-preserving parameter-efficient fine-tuning of AI foundation models," *arXiv preprint arXiv:2410.00433*, 2024.

[54] J. Frery, R. Bredehoft, J. Klemsa, A. Meyre, and A. Stoian, "Private loRA fine-tuning of open-source LLMs with homomorphic encryption," *arXiv preprint arXiv:2505.07329*, 2025.

[55] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019. [Online]. Available: https://arxiv.org/abs/1902.11175

[56] J. Wang *et al.*, "Towards one-shot federated learning: Advances, challenges, and future directions," *arXiv preprint arXiv:2505.02426*, 2025.

[57] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," *arXiv preprint arXiv:1910.03581*, 2019. [Online]. Available: https://arxiv.org/abs/1910.03581

[58] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, "DS-FL: Distillation-based semi-supervised federated learning for medical image classification," in *IEEE ICC*, 2021.

[59] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *NeurIPS*, vol. 33, 2020. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/18df51b97ccd68128e994804f3eccc87-Abstract.html

[60] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, vol. 35, 2022, pp. 21 414–21 428. [Online]. Available: https://arxiv.org/abs/2112.12371

[61] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.

[62] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.

[63] X. Liu, L. Liu, F. Ye, Y. Shen, X. Li, L. Jiang, and J. Li, "FedLPA: One-shot federated learning with layer-wise posterior aggregation," in *NeurIPS*, 2024.

[64] Y. Zhang *et al.*, "Federated one-shot learning with data-free distillation," in *NeurIPS*, 2024.

[65] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.

[66] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025, pp. 2601–2610. [Online]. Available: https://openaccess.thecvf.com/content/WACV2025/html/Mendieta_Navigating_Heterogeneity_and_Privacy_in_One-Shot_Federated_Learning_with_Diffusion_Models_WACV_2025_paper.html

[67] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021, pp. 1484–1490.

[68] L. Sun and L. Lyu, "Federated model distillation with noise-free differential privacy," in *IJCAI*, 2021, pp. 1563–1569.

[69] F. Tramèr and D. Boneh, "Differentially private learning needs better features (or much more data)," in *International Conference on Learning Representations (ICLR)*, 2021, arXiv:2011.11660.

[70] H. Mehta, W. Krichene, A. Thakurta, A. Kurakin, and A. Cutkosky, "Differentially private image classification from features," *Transactions on Machine Learning Research (TMLR)*, 2023, arXiv:2211.13403.

[71] D. Yu, S. Naik, A. Backurs, S. Gopi, H. A. Inan, G. Kamath, J. Kulkarni, Y. T. Lee, A. Manoel, L. Wutschitz *et al.*, "Differentially private fine-tuning of language models," in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2110.06500.

[72] X. Li, F. Tramèr, P. Liang, and T. Hashimoto, "Large language models can be strong differentially private learners," in *International Conference on Learning Representations (ICLR)*, 2022, arXiv:2110.05679.

[73] X.-Y. Liu, R. Zhu, D. Zha, J. Gao, S. Zhong, M. White, and M. Qiu, "Differentially private low-rank adaptation of large language model using federated learning," *arXiv preprint arXiv:2312.17493*, 2023.

[74] J. Xu, K. Saravanan, R. van Dalen, H. Mehmood, D. Tuckey, and M. Ozay, "DP-DyLoRA: Fine-tuning transformer-based models on-device under differentially private federated learning using dynamic low-rank adaptation," *arXiv preprint arXiv:2405.06368*, 2024.

[75] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, G. Wang, and Y. Chen, "Towards building the federated GPT: Federated instruction tuning," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, arXiv:2305.05644 (FedIT).

[76] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, and A. Li, "FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2409.05976.
[77] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.11505 (FlexLoRA).
[78] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, and G. Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, arXiv:2401.06432.
[79] P. Guo, S. Zeng, Y. Wang, H. Fan, F. Wang, and L. Qu, "Selective aggregation for low-rank adaptation in federated learning," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.01463 (FedSA-LoRA).
[80] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *International Conference on Learning Representations (ICLR)*, 2023.
[81] Z. Tao, I. Mason, S. Kulkarni, and X. Boix, "Task arithmetic through the lens of one-shot federated learning," *Transactions on Machine Learning Research (TMLR)*, 2025, arXiv:2411.18607.
[82] Anonymous, "Revisiting federated fine-tuning: A single communication round is enough for foundation models," *arXiv preprint arXiv:2412.04650*, 2024.
[83] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017, pp. 409–437.
[84] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.
[85] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *Advances in Cryptology – EUROCRYPT 2018*, ser. Lecture Notes in Computer Science, vol. 10820. Springer, 2018, pp. 360–384. [Online]. Available: https://eprint.iacr.org/2018/153
[86] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.
[87] J. H. Cheon, D. Kim, and D. Kim, "Efficient homomorphic comparison methods with optimal complexity," in *Advances in Cryptology – ASIACRYPT 2020*, ser. Lecture Notes in Computer Science, vol. 12492, 2020, pp. 221–256.
[88] J. Zhang, J. Liu, X. Yang, Y. Wang, K. Chen, X. Hou, K. Ren, and X. Yang, "Secure transformer inference made non-interactive," in *Network and Distributed System Security Symposium (NDSS)*, 2025, cryptology ePrint Archive, Paper 2024/136.
[89] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction APIs," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 601–618.
[90] D. Lowd and C. Meek, "Adversarial learning," in *Proceedings of the 11th ACM SIGKDD International Conference on Knowledge Discovery in Data Mining (KDD)*, 2005, pp. 641–647.
[91] C. A. Choquette-Choo, F. Tramèr, N. Carlini, and N. Papernot, "Label-only membership inference attacks," in *International Conference on Machine Learning (ICML)*, 2021, pp. 1964–1974.
[92] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2007, pp. 94–103.
[93] A. Viand, C. Knabenhans, and A. Hithnawi, "SoK: Fully homomorphic encryption compilers and verifiable computation," in *IEEE S&P*, 2023. [Online]. Available: https://arxiv.org/abs/2301.07041
[94] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.
[95] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2022, pp. 965–978.
[96] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.
[97] S. Atapoor, K. Baghery, H. V. L. Pereira, and J. Spiessens, "Verifiable FHE via lattice-based SNARKs," Cryptology ePrint Archive, Paper 2024/582, 2024. [Online]. Available: https://eprint.iacr.org/2024/582

HALİL İBRAHİM KANPAK received his B.Sc. degree in Mathematics from Koç University. He is currently pursuing his Ph.D. at Koç University, where he is a member of the Cryptography, Cyber Security & Privacy Research Group. His research interests include cryptography and related areas in Deep Learning.

Asst. Prof. SİNEM SAV received her Ph.D. in Computer and Communication Sciences from École Polytechnique Fédérale de Lausanne (EPFL). She also holds M.Sc. and B.Sc. degrees in Computer Engineering from Bilkent University, where she currently serves as an Assistant Professor in the Computer Engineering department. Her research interests include privacy-preserving machine learning and applied cryptography.

Prof. ALPTEKİN KÜPÇÜ received his Ph.D. degree from Brown University Computer Science Department in 2010. Since then, he has been working as a faculty member at Koç University, leading the Cryptography, Cyber Security & Privacy Research Group that he founded. He is an ACM and IEEE Senior Member, and a co-founder of Xtinge Technology Inc.